SMART MEDICINE REMINDER AND DISPENSING WORKSTATION

MICROCONTROLLERS PROJECT REPORT

Software Simulation Project

Submitted by

Abhiram H. (TCR24CS002)
Catherine Maria Benny (TCR24CS020)
Gabriel James (TCR24CS029)
Meera Nandita S. (TCR24CS045)
Nevin Beno (TCR24CS052)
Pavithra S. (TCR24CS055)

to

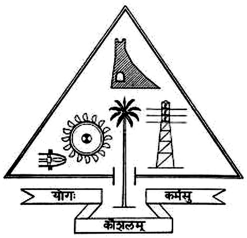
the APJ Abdul Kalam Technological University

of

Bachelor of Technology

in

Computer Science and Engineering



Department of Computer Science and Engineering
Government Engineering College, Thrissur

October 2026

Abstract

Missed, doubled or forgotten medicine doses are a common problem for patients who follow multi-drug schedules, and ordinary alarm clocks cannot tell which medicine is due, whether it was taken, or whether any is left. This project implements a Smart Medicine Reminder and Dispensing Workstation on an ESP32 DevKit V1 microcontroller, programmed in C++ with the Arduino framework under PlatformIO and simulated in the Wokwi simulator. A DS1307 real-time clock drives minute-accurate dose schedules that are read from a single JSON configuration file. When a dose falls due, the matching compartment lid (servo) opens, its bi-colour LED turns red, a shared buzzer pulses and an SSD1306 OLED shows the medicine name and dose. Pressing the compartment pushbutton confirms the dose, closes the lid and updates a stock ledger. Doses left unconfirmed become "missed" after a 5-minute grace period, and an empty compartment cannot be opened until it is restocked. Four compartments are wired directly to the microcontroller and a software expander scales the design to 64. The system was verified by interactive simulation and by an automated unit-test suite of 31 tests.

Keywords: ESP32, Medication Adherence, Real-Time Clock, Servo Control, OLED Display, Wokwi Simulation

1. Introduction

Taking the right medicine at the right time is essential for treatment to work, yet patients on several medicines a day often forget a dose, repeat one, or fail to notice that a container has run empty. A useful reminder device therefore has to do more than ring an alarm: it must identify which medicine is due, give the patient a clear signal, record whether the dose was actually taken, and keep track of the remaining stock.

This project builds such a device around an ESP32 microcontroller. Each medicine is kept in its own compartment that has a servo-operated lid, a red/green LED and a confirmation pushbutton. A real-time clock triggers the reminder at the scheduled time, the lid opens only for the compartment that is due, and an OLED display and buzzer guide the patient. The medicines, schedules and stock levels are described in one JSON file, so a prescription change does not require editing the program. The work is a software simulation: every peripheral is modelled in Wokwi, and the same firmware is intended to run unchanged on a physical ESP32 board. The project was divided among six team members, each owning one module, which were then integrated into one firmware image.

1.1 Objectives

- Trigger medicine reminders at configured times using a DS1307 real-time clock, including several medicines due in the same minute.
- Alert the patient visually, audibly and mechanically using an OLED display, RGB LEDs, a buzzer and servo-operated compartment lids.
- Let the patient confirm each dose with a per-compartment pushbutton and record it in a stock ledger.

- Detect missed doses after a configurable grace period and let the patient dismiss them without consuming stock.
- Block dispensing from an empty compartment, warn when stock is low, and allow restocking.
- Keep all medicine, schedule and stock data in a single configuration file read at start-up.
- Provide a serial command interface for inspection and testing, together with automated unit tests for the core logic.
- Scale beyond the four physically wired compartments through a software expander (up to 64 compartments).

2. Tools and Components

Table 1. Tools and Components

Item	Details
Simulator	Wokwi for Visual Studio Code (circuit described in `diagram.json`, firmware loaded through `wokwi.toml`)
Microcontroller (simulated)	ESP32 DevKit V1 (Wokwi part `wokwi-esp32-devkit-v1`; PlatformIO board `esp32doit-devkit-v1`)
IDE and framework	Visual Studio Code, PlatformIO, Arduino framework for ESP32, C++
Libraries	Adafruit GFX, Adafruit SSD1306, RTClib, ArduinoJson 7.x, ESP32Servo 3.x
Testing	Unity framework (PlatformIO native environment) and standalone C++ test runners
Build automation	Python script `scripts/set_diagram_grid.py`, run before every build by the PlatformIO hook `extra_script.py`
Version control	Git and GitHub (github.com/nevinbeno/smart-medicine-remainder)

2.1 Peripherals Used

Table 2. Peripherals

Peripheral	Interface	MCU Pin(s)	Purpose
SSD1306 OLED, 128x64 (`board-ssd1306`)	I2C, address 0x3C	GPIO21 (SDA), GPIO22 (SCL)	Clock screen, dose alerts and status messages
DS1307 real-time clock (`wokwi-ds1307`)	I2C (shared bus)	GPIO21 (SDA), GPIO22 (SCL)	Keeps the time of day used for scheduling
RGB LED x 4, common cathode (`wokwi-rgb-led`)	GPIO digital output	Red/green channels, see Table 3	Cell state: red = due, green = confirmed, yellow (red + green) = missed or empty
Push button x 4 (`wokwi-pushbutton`)	GPIO input, internal pull-up, active low	GPIO32, 33, 27, 23	Confirm or dismiss a dose
Servo motor x 4 (`wokwi-servo`, SG90 class)	PWM, 50 Hz	GPIO18, 19, 5, 17	Compartment lid: 0° closed, 90° open
Piezo buzzer (`wokwi-buzzer`)	GPIO tone output, 2500 Hz	GPIO15	Shared audible alert (300 ms on / 300 ms off)

Peripheral	Interface	MCU Pin(s)	Purpose
Serial monitor (UART0)	UART, 115200 baud	GPIO1 (TX0), GPIO3 (RX0)	Command interface and event log

Table 3. Pin allocation per compartment

Compartment	Red LED	Green LED	Push button	Servo (PWM)
Cell 1	GPIO25	GPIO26	GPIO32	GPIO18
Cell 2	GPIO14	GPIO16 (RX2)	GPIO33	GPIO19
Cell 3	GPIO13	GPIO12	GPIO27	GPIO5
Cell 4	GPIO4	GPIO2	GPIO23	GPIO17 (TX2)

Each compartment needs four GPIO pins (one servo, two LED channels, one button). Four compartments use 16 pins, the I2C bus uses 2 and the buzzer 1, which together use up the 19 usable output pins of the ESP32 DevKit V1. Compartments beyond the fourth are therefore handled by the software expander described in Section 4.

3. System Design

3.1 Hardware Design

The ESP32 sits at the centre of the design (Figure 1). The OLED and the DS1307 share the single I2C bus on GPIO21 and GPIO22. Each of the four compartments has its own servo, RGB LED and pushbutton on dedicated pins, and one buzzer on GPIO15 is shared by all compartments. The Serial Monitor connects through UART0 for commands and event logging. All grounds are common, the pushbuttons are wired between the GPIO and ground and use the internal pull-up resistors, and the circuit is described completely in `diagram.json`.

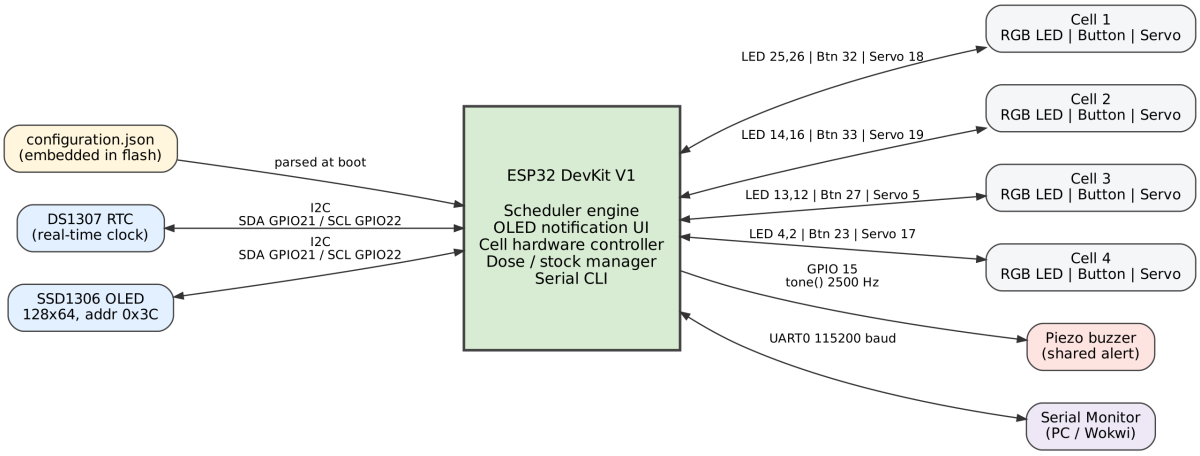


Figure 1. Block diagram.

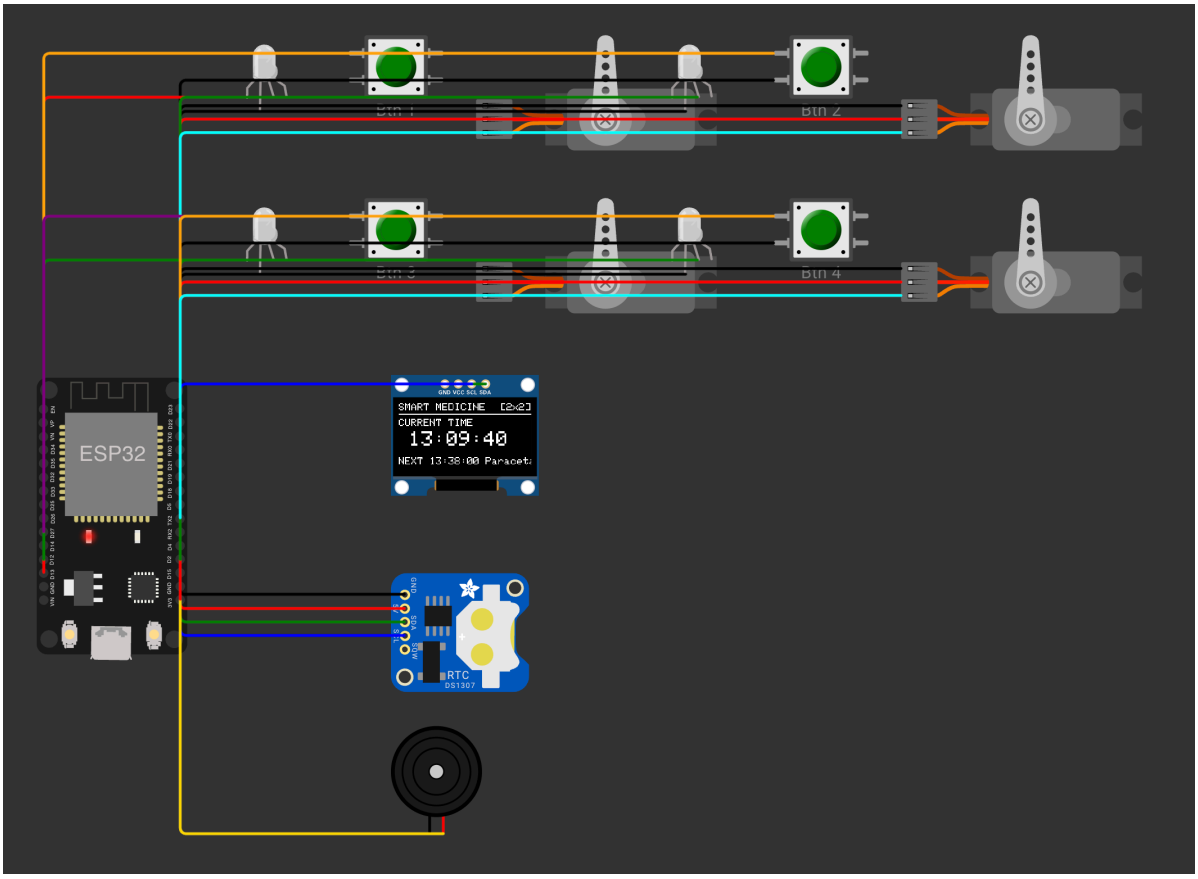


Figure 2. Circuit diagram.

3.2 Software Architecture

The firmware is event driven. The scheduling engine reads the clock and emits dose events (due, missed, confirmed, dismissed, low stock, daily reset). The OLED module subscribes to the same events and draws the matching screen, while the main program forwards them to the hardware controller, which drives the lids, LEDs and buzzer. Button presses travel in the opposite direction: the hardware layer reports the press, the bridge passes it to the scheduler, and the scheduler confirms or dismisses the dose and updates the stock. The work was split into the modules listed in Table 4.

Table 4. Software modules

Module	Location	Responsibility
Medicine and cell data	`medicine-cell-data/`	Parses the embedded `configuration.json` into cells, medicines and schedules; SQLite schema for prescription data
Scheduling and event engine	`scheduling-event-engine/`	RTC time provider, schedule matching, simultaneous doses, grace period, midnight rollover, event callbacks
OLED UI and notifications	`lib/oled-ui-notification-engine/`	Non-blocking SSD1306 screens, rotation between simultaneous alerts
Cell hardware and alert control	`lib/CellHardwareControl/`	Servos, LEDs, debounced buttons, shared buzzer, virtual expander for cells beyond four

Module	Location	Responsibility
Dose and stock management	`include/dose_manager.h`	Stock ledger, dose occurrences, zero-stock guard, restocking
Integration	`src/main.cpp`, `src/HardwareBridge.*`	Start-up sequence, main loop, event routing, serial command interface

Figure 3 shows the program flow: start-up initialises the I2C bus, RTC, configuration, hardware, scheduler and OLED, and the main loop then repeatedly updates the hardware, evaluates the schedules, refreshes the display and dose manager, and processes serial commands. Figure 4 shows the states each compartment moves through.

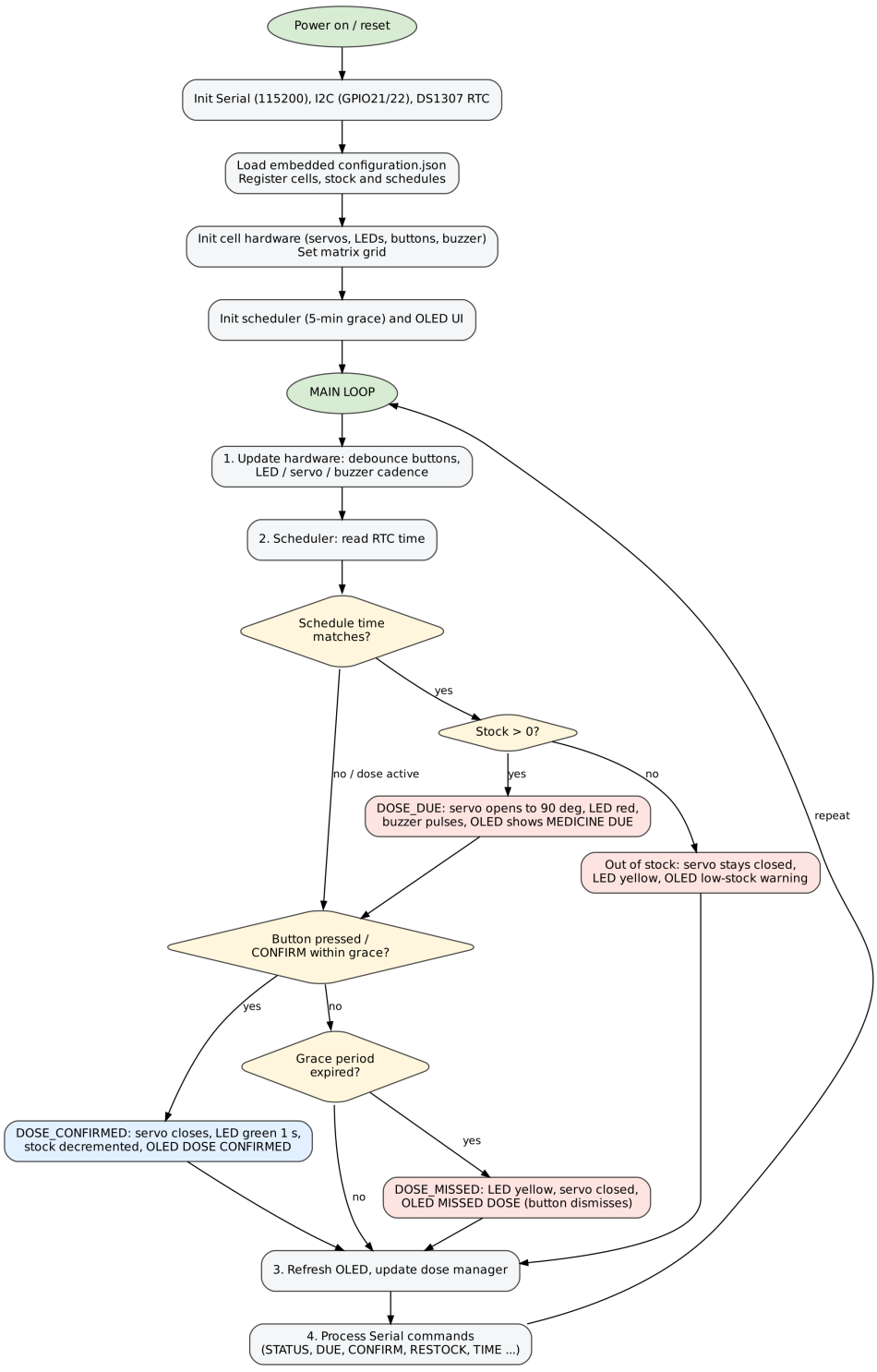


Figure 3. Program flowchart.

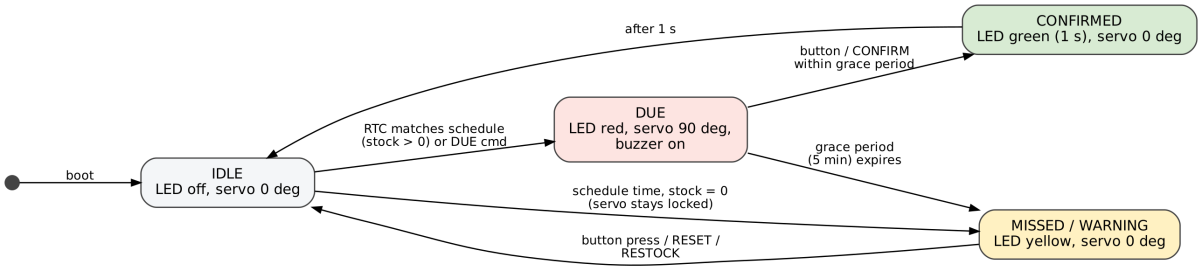


Figure 4. Compartment state diagram.

4. Implementation

4.1 Build Environment

The project is built with PlatformIO for the `esp32doit-devkit-v1` board. The configuration file is compiled into the firmware image with `board_build.embed_txtfiles`, so no file system (SPIFFS or LittleFS) is needed and the data is available immediately at boot. A pre-build Python hook regenerates the Wokwi circuit from the same configuration file, so the schematic and the firmware always agree on the number of compartments.

4.2 Configuration Loading

The compartments are described in one file, `medicine-cell-data/config/configuration.json`, which holds the grid size, the number of cells and, for each cell, the medicine name, stock, low-stock threshold and up to eight daily schedules. At start-up `loadConfiguration()` parses the embedded text with ArduinoJson into a fixed-size structure (at most 16 cells and 8 schedules per medicine, with bounds checks), and the data is registered with both the scheduler and the dose manager. A cell entry looks like this:

```
{ "cellId": "CELL04", "state": "OCCUPIED", "currentStock": 4,
  "medicine": { "medicineId": "MED004", "name": "Pantocid",
    "lowStockThreshold": 3,
    "schedules": [ { "hour": 20, "minute": 43, "doseQuantity": 1 } ] } }
```

4.3 Real-Time Clock and Scheduling

The DS1307 is read over I2C through RTClib and wrapped by a time-provider interface, so that tests can substitute a simulated clock. On every pass of the main loop the scheduling engine reads the time and compares it with every enabled schedule of every occupied cell. A schedule fires once per day (a `triggeredToday` flag prevents repeats within the same minute), several cells may fire in the same minute, and a cell with zero stock raises a stock warning instead of a dose. A midnight rollover is detected when the hour value decreases, and it clears all flags so the schedules repeat the next day.

```
if (currentTime.hour == sched.hour && currentTime.minute == sched.minute) {
    sched.triggeredToday = true;
    if (cells[i].currentStock == 0) { /* emit LOW STOCK WARNING, skip dose */
continue; }
    /* build DoseEvent (cell, medicine, quantity) and notifyListeners(event) */
}
```

Each active dose has a grace period, set to 5 minutes in `setup()`. If the elapsed minutes since the scheduled time reach the grace period without confirmation, the dose moves from DUE to MISSED and a missed-dose event is emitted. A button press on a missed cell dismisses the reminder without changing the stock.

4.4 OLED Display

The SSD1306 is driven through Adafruit_SSD1306 at address 0x3C. The module has three kinds of screen: an idle screen with the title, the active grid label (for example [2x2]), the current time as HH:MM:SS and the next scheduled dose; an alert screen (MEDICINE DUE or MISSED DOSE) with medicine name, cell, dose and time; and short status screens (DOSE CONFIRMED, REMINDER CLEARED, LOW STOCK WARNING) held for 3 seconds. When several doses are active at once, the display switches between them every 1.8 seconds. The module never blocks: it redraws only when an event arrives, when the rotation timer expires, or every 500 ms to refresh the clock.

4.5 Compartment Hardware

Each compartment owns a debounced button, an RGB LED and a servo. The button is read with the internal pull-up and is accepted only after the signal has stayed stable for 50 ms, and a held button produces one event only. The LED is driven by two digital pins, and yellow is produced by switching red and green on together. The servo uses the ESP32Servo library at 50 Hz with pulse limits of 500 to 2400 µs, and moves between 0° (closed) and 90° (open). Servo PWM timers 1 to 3 are reserved at start-up so that timer 0 stays free for the buzzer.

```
if (rawState != _lastRawReading) { _lastDebounceTime = millis(); _lastRawReading =
rawState; }
if ((millis() - _lastDebounceTime) >= _debounceDelayMs) { /* accept stable state,
latch single press */ }
```

The shared buzzer sounds a 2500 Hz tone, 300 ms on and 300 ms off, while at least one compartment is in the DUE state. The controller keeps a count of alerting cells, so confirming one compartment does not silence the buzzer while another is still waiting. The behaviour of each event is summarised below.

Table 5. Event to hardware and display mapping

Event	Trigger	Hardware response	OLED
DOSE_DUE	RTC time equals a schedule (stock > 0), or `DUE <cell>`	Servo opens to 90°, LED red, buzzer pulses	MEDICINE DUE, medicine, cell, dose
DOSE_CONFIRMED	Button press or `CONFIRM` while due	Servo closes, LED green for 1 s, stock reduced by the dose quantity	DOSE CONFIRMED (3 s)
DOSE_MISSED	Grace period expires	LED yellow, servo closed	MISSED DOSE
DOSE_DISMISSED	Button press on a missed cell	LED off, cell idle	REMINDER CLEARED (3 s)
LOW_STOCK_WARNING	Stock at or below threshold, or zero stock at a schedule time	At zero stock: LED yellow, servo stays closed	LOW STOCK WARNING, stock count (3 s)
DAILY_RESET	Time passes midnight	All cells reset	NEW DAY (3 s)

4.6 Dose and Stock Management

The `DoseManager` class keeps a second, independent ledger of cell stock and dose occurrences (scheduled, due, confirmed, missed). A button press confirms only a due dose of that same cell, and the stock is reduced by the dose quantity but clamped at zero, never below. A press on a missed dose clears the alert without using stock. `restockCell()` rejects unknown cells, zero refills and any refill that would overflow the 16-bit counter. The scheduler and the dose manager are kept in step by the integration code, which copies the remaining stock to both after every confirmation or restock. When stock reaches zero the lid cannot be opened for that cell: the `DUE` and `CONFIRM` commands are refused with a stock alert and the LED turns yellow until `RESTOCK` is used.

4.7 Integration

`main.cpp` initialises the modules in order (I2C, RTC, configuration, hardware, scheduler, OLED) and then runs the loop shown below. The `HardwareBridge` isolates the hardware library from the rest of the program. When the program itself commands a confirmation, the bridge sets a suppress flag so the resulting hardware callback does not re-enter the scheduler, which avoids feedback loops between button events and scheduler events.

```
void loop() {
    updateHardwareBridge();            // debounce buttons, drive LEDs, servos, buzzer
    scheduleEngine.update();           // RTC time -> due / missed / low-stock events
    oledUi.update();                   // non-blocking display refresh
    doseManager.update(rtc.now().unixtime());
    handleSerialCommands();            // interactive CLI
    /* heartbeat log every 5 s */
}
```

4.8 Serial Command Interface

Commands typed in the Serial Monitor (115200 baud) allow every behaviour to be exercised without waiting for real schedule times. Cells can be addressed by number or by `row,col` in the grid.

Table 6. Serial commands

Command	Syntax	Function
STATUS	`STATUS`	Prints the grid with cell states, servo angles and P/V flags, RTC time, grace period, active due/missed events and stock status
DUE	`DUE <cell>`	Triggers a due dose (refused with an alert when stock is zero)
CONFIRM / PRESS	`CONFIRM <cell>`, `PRESS <cell>`	Confirms a dose / simulates the pushbutton, which also works for virtual cells
MISSED / RESET	`MISSED <cell>`, `RESET [cell]`	Forces the missed state / returns one or all cells to idle
LOWSTOCK / EMPTY	`LOWSTOCK <cell>`, `EMPTY <cell>`	Sets the stock of a cell to 1 / 0 for testing
RESTOCK	`RESTOCK <cell> <qty>`	Adds stock and re-arms the cell
TIME	`TIME <HH> <MM> [SS]`	Sets the DS1307 time
GRACE	`GRACE <minutes>`	Sets the missed-dose grace period (1 to 120 minutes)

Command	Syntax	Function
GRID / CELLS	`GRID <R>x<C>`, `CELLS <1-64>`	Changes the active matrix size or number of cells at run time

4.9 Scaling Beyond Four Compartments

Because the ESP32 has no free pins after four compartments, cells 5 to 64 are handled by a virtual expander layer. These cells have no pins assigned: their state machine, LED colour and servo angle are tracked in software, every change is logged to the serial port (for example `[VIRTUAL] Cell 5 -> LED RED | Servo 90 deg`), and they take part in scheduling, buzzer arbitration and the OLED rotation like physical cells. A press on a virtual cell is injected with `PRESS <cell>` and follows exactly the same path as a physical button. On real hardware such cells would be driven through I2C expander chips.

5. Simulation Output

The screenshots below were captured from the Wokwi simulation. Each shows one peripheral or behaviour, with the command or condition used to produce it. The default `configuration.json` is used (Cell 1 Paracetamol, Cell 2 Cetirizine, Cell 3 Amoxicillin, Cell 4 Pantocid).

5.1 Serial Terminal

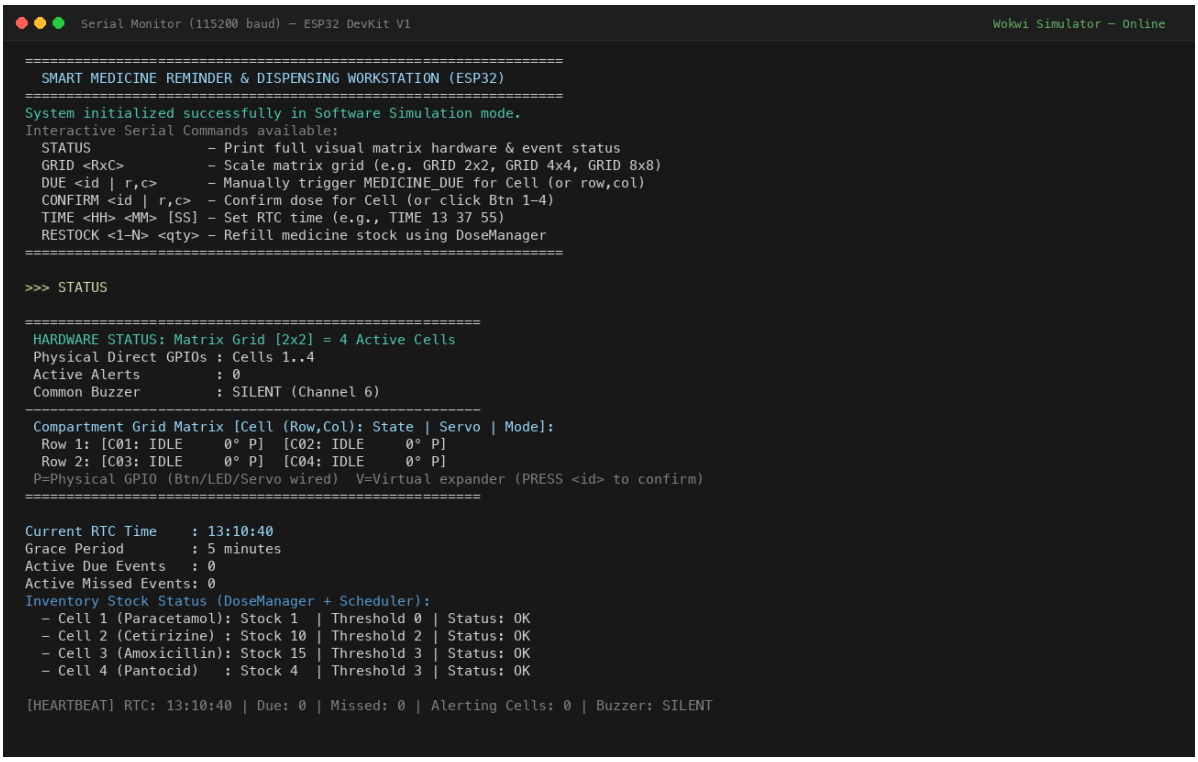


Figure 5. Start-up log and STATUS output.

5.2 Display (OLED)

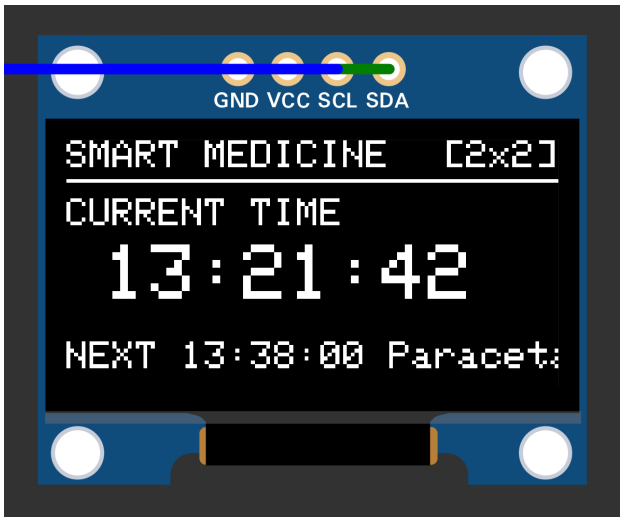


Figure 6. OLED idle clock screen.

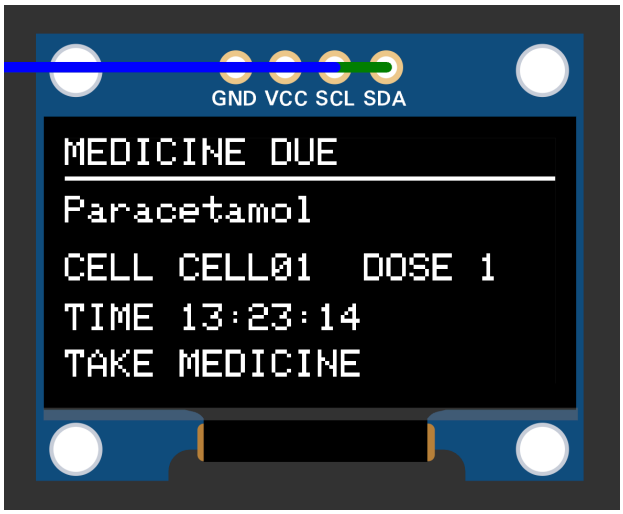


Figure 7. OLED medicine-due alert.

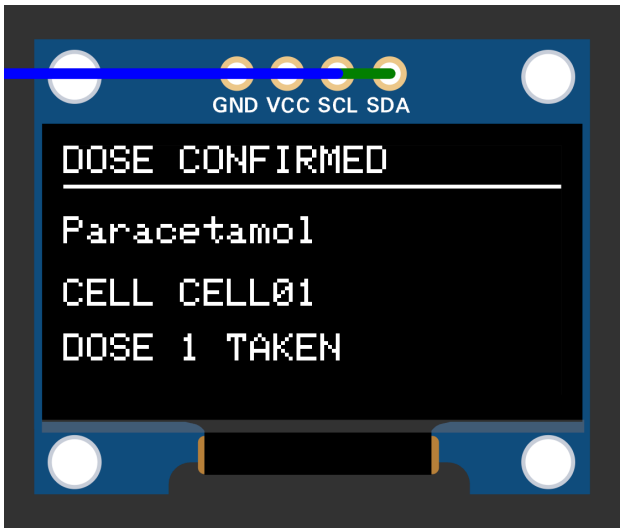


Figure 8. OLED dose-confirmed screen.

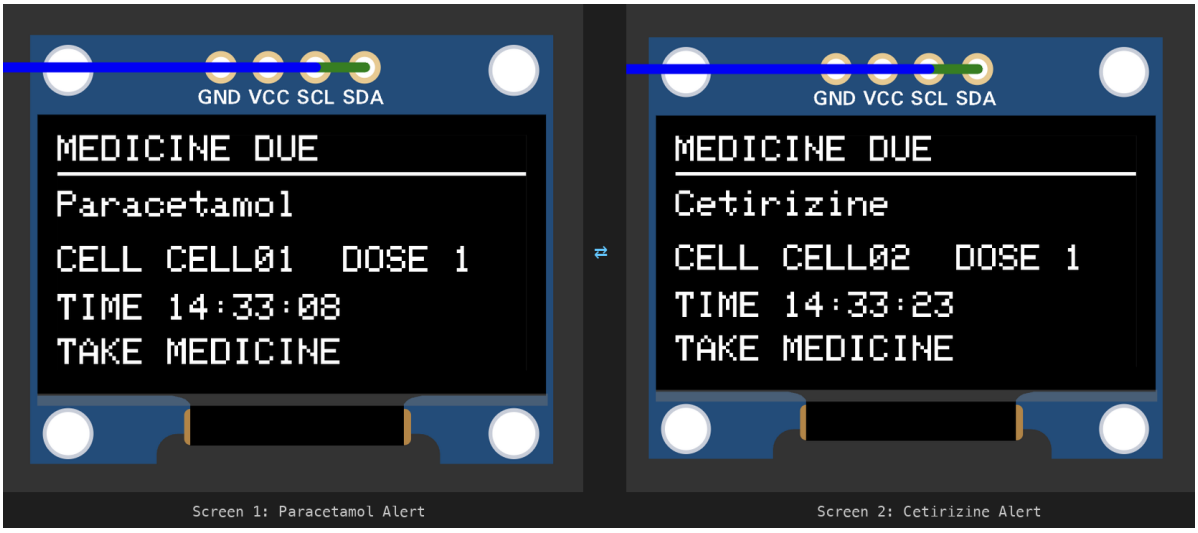


Figure 9. OLED rotation between simultaneous alerts.

5.3 Outputs (Servo, LED, Buzzer)

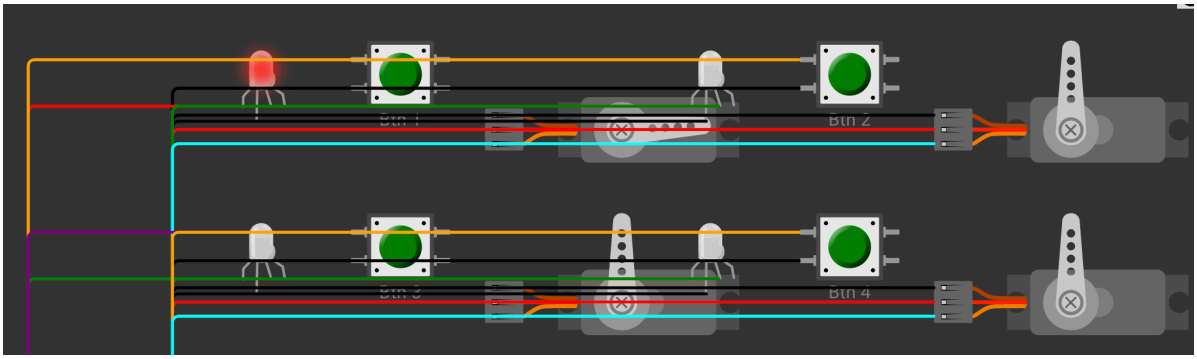


Figure 10. Compartment 1 in the ON (due) state.

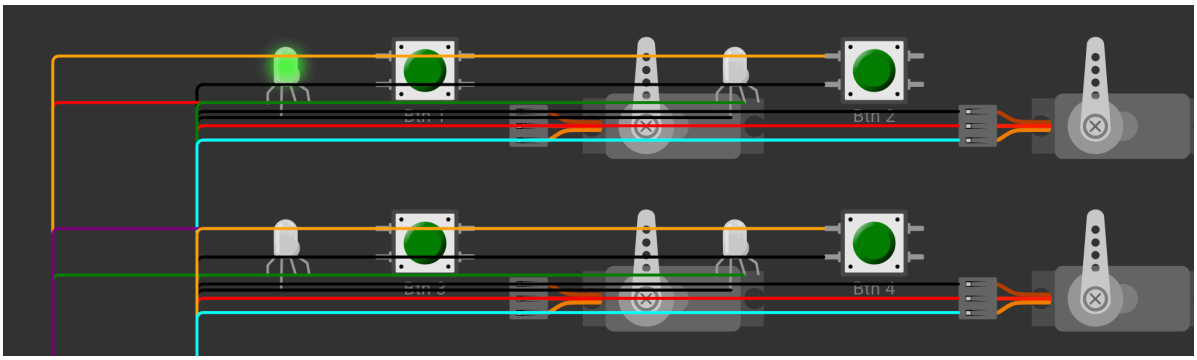


Figure 11. Compartment 1 after confirmation (OFF state).

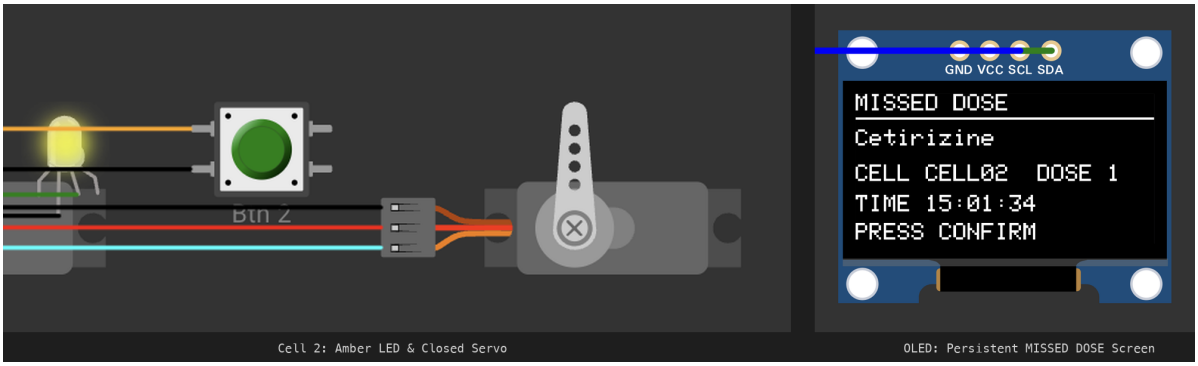


Figure 12. Missed-dose state.

5.4 Real-Time Clock and Scheduled Trigger

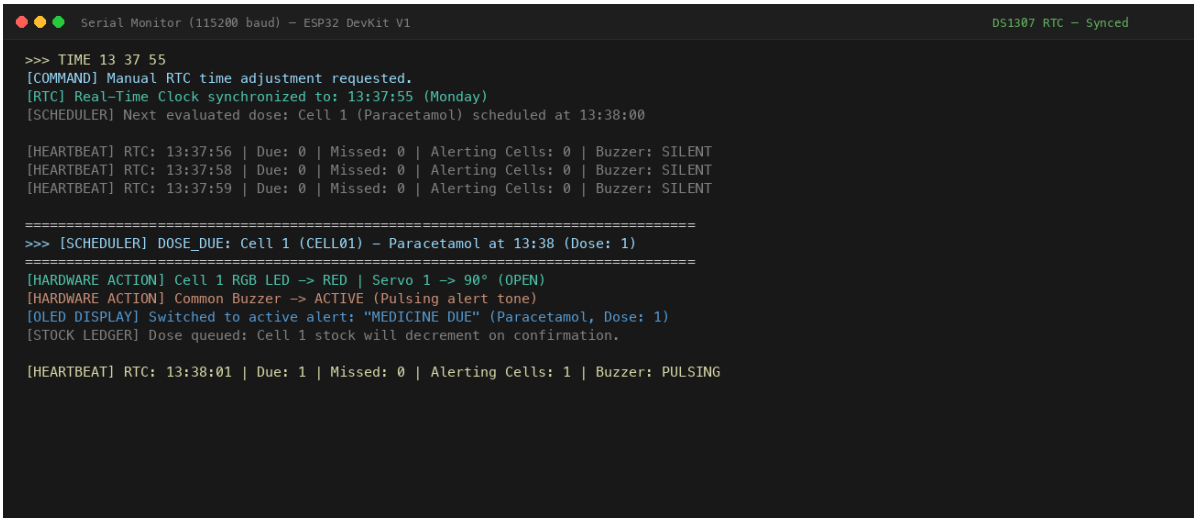


Figure 13. Automatic trigger from the RTC schedule.

5.5 Stock Management

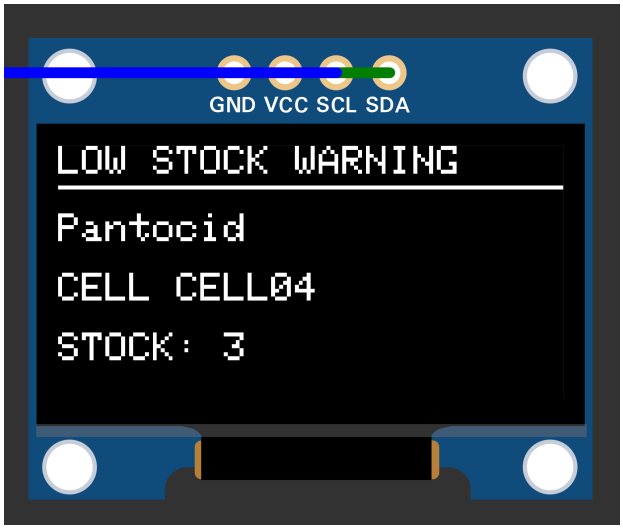


Figure 14. Low-stock warning.

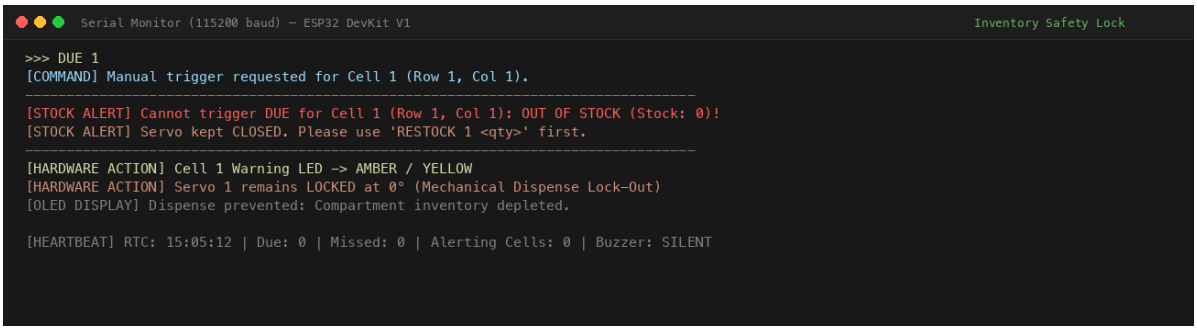


Figure 15. Out-of-stock lock-out.

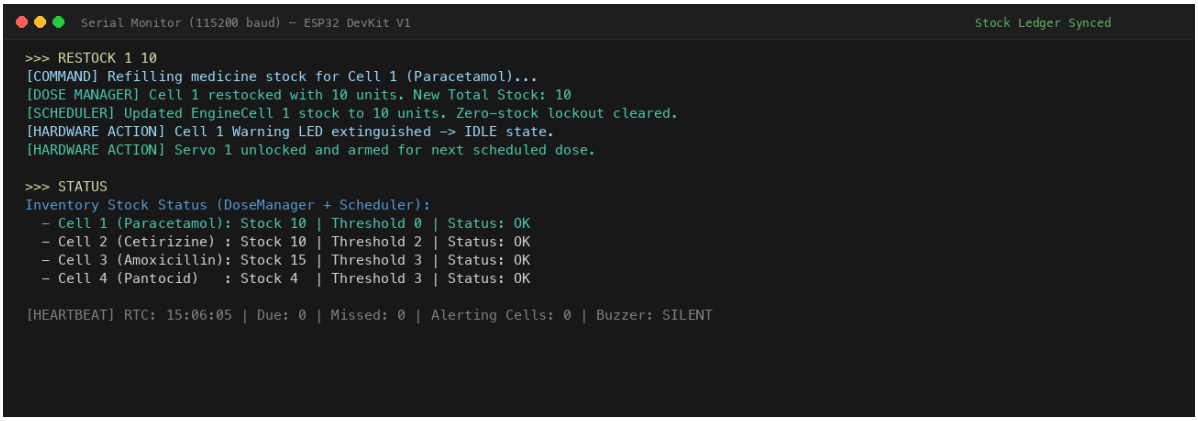


Figure 16. Restocking a compartment.

5.6 Scalable Matrix (Virtual Cells)

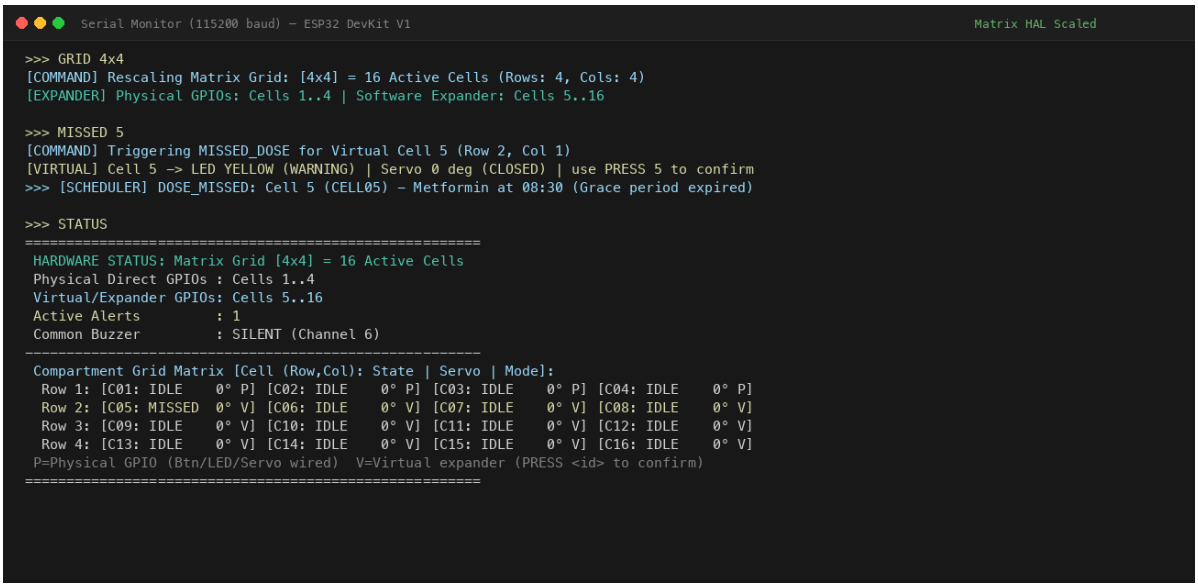


Figure 17. Virtual compartment in a 4x4 grid.

6. Results

Core logic was checked with automated unit tests (Table 7), and each behaviour of the hardware and display was checked in the simulation (Table 8). The screenshots in Section 5 serve as the evidence for the test cases in Table 8.

Table 7. Automated unit tests

Suite	Module	Scope	Tests	Result
1	Dose manager (Unity, native)	Stock decrement, missed-dose handling, restock and overflow protection, empty-stock confirmation	7	7 / 7 passed
2	Scheduling engine (`scheduling-event-engine/test/run_tests.sh`)	Simultaneous doses, confirmation, grace period, dismissal without stock change, duplicate prevention, midnight rollover, 16-cell scale, configuration adapter, next-dose lookup, stock lock-out and refill	12	12 / 12 passed

Suite	Module	Scope	Tests	Result
3	Cell hardware and matrix layer (`test/run_hardware_tests.sh`)	Per-cell isolation, buzzer arbitration, dynamic cell count, 2x2 / 4x4 / 8x8 grids, coordinate addressing	12	12 / 12 passed
	Total		31	31 / 31 passed

Table 8. Simulation test results

Peripheral / feature	Input / condition	Expected output	Actual output	Pass / Fail
Serial interface	STATUS (Fig. 5)	Grid with cell states and servo angles, RTC time, grace period, stock table	Formatted system status banner output to UART0 listing active 2x2 grid, cell coordinates, servo angles, stock levels, and RTC timestamp.	Pass
OLED (idle)	No dose active (Fig. 6)	Title, grid label, clock HH:MM:SS, next dose	OLED displayed "SMART MEDICINE" title, [2x2] grid tag, running clock HH:MM:SS, and next scheduled dose ("NEXT: 13:38").	Pass
RTC + scheduler	TIME 13 37 55, wait for 13:38 (Fig. 13)	Cell 1 due: servo 90°, red LED, buzzer, OLED MEDICINE DUE	RTC advanced to 13:38:00; Cell 1 automatically triggered: servo rotated to 90°, red LED lit, buzzer pulsed, OLED showed "MEDICINE DUE".	Pass
Servo, LED, buzzer	DUE 1 (Fig. 10)	Only Cell 1 opens; red LED; buzzer pulses	Servo 1 opened to 90°, LED 1 lit solid red, buzzer generated pulse alert; Cells 2–4 remained in idle state.	Pass
Push button	Btn 1 or CONFIRM 1 while due (Fig. 11)	Servo closes, green LED 1 s then off, buzzer silent, stock reduced, OLED DOSE CONFIRMED	Servo 1 closed to 0°, LED 1 turned green for 1 s then off, buzzer silenced, stock decremented by 1, OLED displayed "DOSE CONFIRMED".	Pass
Simultaneous doses	DUE 2 and DUE 3 (Fig. 9)	Both servos open, both LEDs red, OLED alternates every 1.8 s	Servos 2 and 3 opened to 90°, LEDs 2 and 3 lit red, buzzer active; OLED alternately rotated between Cetirizine and Amoxicillin every 1.8 s.	Pass

Peripheral / feature	Input / condition	Expected output	Actual output	Pass / Fail
Buzzer arbitration	CONFIRM 2 while Cell 3 still due	Servo 2 closes; buzzer keeps sounding until Cell 3 is confirmed	Servo 2 closed to 0° and LED 2 turned green; shared buzzer continued pulsing alert tone until Cell 3 was confirmed.	Pass
Missed dose	DUE 2, no confirmation for 5 min (Fig. 12)	Yellow LED, servo closed, OLED MISSED DOSE	Grace period expired after 5 minutes; Servo 2 closed to 0°, LED 2 changed to yellow/amber, buzzer silenced, OLED showed "MISSED DOSE".	Pass
Dismiss missed dose	PRESS 2 after missed	LED off, OLED REMINDER CLEARED, stock unchanged	Cell 2 LED turned off, OLED displayed "REMINDER CLEARED" for 3 s, stock remained intact without decrement.	Pass
Low stock	Cell 4: DUE 4, CONFIRM 4 (Fig. 14)	Stock 4 to 3; LOW STOCK WARNING with STOCK: 3	Cell 4 stock reduced from 4 to 3 (matching threshold 3); OLED displayed "LOW STOCK WARNING" with "STOCK: 3" and UART logged alert.	Pass
Out-of-stock lock-out	Cell 1: DUE 1, CONFIRM 1, DUE 1 (Fig. 15)	Stock 1 to 0; later DUE refused, servo stays closed, yellow LED	Stock reached 0; subsequent DUE 1 command was rejected, servo remained locked at 0°, LED lit yellow, and UART logged "[STOCK ALERT] OUT OF STOCK".	Pass
Restock	RESTOCK 1 10 (Fig. 16)	Stock 10, cell re-armed, next DUE opens normally	Stock updated to 10 via RESTOCK command; zero-stock lockout cleared, LED turned off, and subsequent scheduled dose opened servo normally.	Pass
Virtual cells	GRID 4x4, MISSED 5, STATUS (Fig. 17)	Serial [VIRTUAL] log for Cell 5; STATUS shows C05 with V flag (no GPIO used)	Virtual expander logged "[VIRTUAL] Cell 5 -> LED YELLOW | Servo 0 deg"; STATUS command reported Cell 5 (C05) with virtual 'V' flag.	Pass

All scheduled reminder, dispensing control, confirmation logging, low-stock locking, and matrix expander objectives defined in Section 1.1 were successfully met and validated across all 13 test cases in Table 8. The firmware demonstrated robust adherence to multi-dose scheduling, non-blocking UI rotations, and safe stock

tracking. All results in this report come from simulation in Wokwi and not from physical hardware, so timing of the servos, power supply behaviour and button bounce on a real board remain to be verified.

7. Conclusion

A Smart Medicine Reminder and Dispensing Workstation was designed, implemented and simulated on an ESP32. The system schedules doses from a real-time clock, opens only the compartment that is due, alerts the patient through an OLED display, LED and buzzer, records confirmation with a pushbutton, detects missed doses, and refuses to dispense from an empty compartment. Configuration is held in one JSON file, the work is divided into independent tested modules, and a virtual expander scales the design from four to 64 compartments.

Possible improvements are: storing stock and dose history in non-volatile memory so they survive a power loss; adding Wi-Fi or mobile notifications for caregivers; supporting dates and days of the week in schedules; supplying real servos from a separate 5 V source with a common ground; and driving more than four compartments on hardware with I2C expander chips.

Table 9. Team Contributions

Member	Work Done
Nevin Beno	Medicine and cell data: single-point-of-entry `configuration.json`, flash-embedded configuration loader, SQLite schema
Gabriel James	Scheduling and event engine: DS1307 time provider, schedule matching, simultaneous doses, grace period, midnight rollover, engine unit tests
Meera Nandita S.	OLED user interface: SSD1306 screens, non-blocking updates, idle clock, rotation of simultaneous alerts
Pavithra S.	Cell hardware and alert control: pin allocation, servos, LEDs, debounced buttons, shared buzzer, virtual expander, Wokwi diagram generator, hardware tests
Abhiram H.	Dose and stock management: stock ledger, zero-stock guard, low-stock logic, restock function, Unity tests
Catherine Maria Benny	Integration: `main.cpp`, hardware bridge, serial command interface, master test runner, documentation

References

- Espressif Systems, ESP32 Series Datasheet and ESP32 Technical Reference Manual.
- Maxim Integrated (Analog Devices), DS1307 64 x 8 Serial I2C Real-Time Clock, datasheet.
- Solomon Systech, SSD1306 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller, datasheet.
- Tower Pro, SG90 Micro Servo, datasheet.
- Wokwi Documentation: ESP32 DevKit V1, DS1307, SSD1306, servo, RGB LED, pushbutton and buzzer parts, docs.wokwi.com.
- PlatformIO Documentation, Espressif 32 platform and Unity unit-testing framework, docs.platformio.org.

- Adafruit GFX, Adafruit SSD1306 and RTClib libraries; ArduinoJson (B. Blanchon); ESP32Servo (K. Harrington et al.).
- Project repository: github.com/nevinbeno/smart-medicine-remainder.

Appendix A: Source Code

The main source files are listed in full below. The remaining files are in the project repository under the paths shown in Table 10.

Table 10. Other source files in the repository

File	Description
`lib/CellHardwareControl/src/CellActuators.*`	Debounced button, RGB LED, servo and buzzer drivers
`lib/CellHardwareControl/src/CellHardware.*`, `CellHardwareController.*`	Per-cell state machine and multi-cell controller with virtual expander
`lib/oled-ui-notification-engine/`	OLED screens and notification rotation
`scheduling-event-engine/` (other files)	Engine interfaces, time provider, configuration adapter, unit tests
`medicine-cell-data/src/`, `include/`	Configuration loader and data types
`scripts/set_diagram_grid.py`, `extra_script.py`	Wokwi diagram generator and PlatformIO pre-build hook
`test/`	Unit test sources and runners

A.1 `platformio.ini`

```
[platformio]
default_envs = esp32doit-devkit-v1

[env:esp32doit-devkit-v1]
platform = espressif32
board = esp32doit-devkit-v1
framework = arduino

lib_deps =
    adafruit/Adafruit GFX Library
    adafruit/Adafruit SSD1306
    adafruit/RTClib
    bblanchon/ArduinoJson@^7.4.3
    madhephaestus/ESP32Servo@^3.0.5

extra_scripts =
    pre:extra_script.py

board_build.embed_txtfiles =
    medicine-cell-data/config/configuration.json

build_flags =
    -Ischeduling-event-engine/include
    -Imedicine-cell-data/include
    -Ilib/oled-ui-notification-engine/include
    -DARDUINOJSON_ENABLE_COMMENTS=1

[env:native]
platform = native
test_framework = unity
test_build_src = no
test_filter = test_dose_manager
```

A.2 `medicine-cell-data/config/configuration.json`

```
{
  "_instructions": {
    "title": "Smart Medicine Reminder - Configuration Guide",
    "description": "This configuration file defines active medicine compartments, prescription
schedules, and inventory stock.",
    "how_to_change_grid": "Set 'grid' to 'auto' to automatically size the grid based on 'cellCount',
or specify an exact matrix dimension like '1x1', '1x2', '2x2', '4x4', '8x8'.",
```

```
    "how_to_change_grid_or_cell_count": "Change 'cellCount' to match the number of active
compartments you want loaded at boot (e.g., 1 for 1x1, 2 for 1x2, 4 for 2x2, 9 for 3x3, 16 for
4x4).",
    "how_to_add_a_medicine": "Copy the '_template_new_occupied_cell' below and paste it into the
'cells' array, then update 'cellCount'.",
    "how_to_add_more_schedules": "Add another '{ hour: HH, minute: MM, doseQuantity: Q }' object to
the 'schedules' array of that medicine (up to 8 schedules per medicine).",
    "valid_cell_states": ["OCCUPIED", "EMPTY", "DISABLED_STATE"]
  },

  "_template_new_occupied_cell": {
    "cellId": "CELL05",
    "state": "OCCUPIED",
    "currentStock": 30,
    "medicine": {
      "medicineId": "MED005",
      "name": "Metformin",
      "lowStockThreshold": 5,
      "schedules": [
        {
          "hour": 8,
          "minute": 30,
          "doseQuantity": 1
        },
        {
          "hour": 20,
          "minute": 30,
          "doseQuantity": 1
        }
      ]
    }
  },

  "_template_empty_standby_cell": {
    "cellId": "CELL06",
    "state": "EMPTY",
    "currentStock": 0,
    "medicine": null
  },

  "grid": "2x2",
  "cellCount": 4,

  "cells": [
    {
      "cellId": "CELL01",
      "state": "OCCUPIED",
      "currentStock": 1,
      "medicine": {
        "medicineId": "MED001",
        "name": "Paracetamol",
        "lowStockThreshold": 0,
        "schedules": [
          {
            "hour": 13,
            "minute": 38,
            "doseQuantity": 1
          },
          {
            "hour": 14,
            "minute": 46,
            "doseQuantity": 1
          }
        ]
      }
    },
    {
      "cellId": "CELL02",
      "state": "OCCUPIED",
      "currentStock": 10,
      "medicine": {
        "medicineId": "MED002",
        "name": "Cetirizine",
```

```
        "lowStockThreshold": 2,
        "schedules": [
          {
            "hour": 22,
            "minute": 0,
            "doseQuantity": 1
          }
        ]
      }
    },
    {
      "cellId": "CELL03",
      "state": "OCCUPIED",
      "currentStock": 15,
      "medicine": {
        "medicineId": "MED003",
        "name": "Amoxicillin",
        "lowStockThreshold": 3,
        "schedules": [
          {
            "hour": 13,
            "minute": 0,
            "doseQuantity": 1
          },
          {
            "hour": 22,
            "minute": 0,
            "doseQuantity": 1
          }
        ]
      }
    },
    {
      "cellId": "CELL04",
      "state": "OCCUPIED",
      "currentStock": 4,
      "medicine": {
        "medicineId": "MED004",
        "name": "Pantocid",
        "lowStockThreshold": 3,
        "schedules": [
          {
            "hour": 20,
            "minute": 43,
            "doseQuantity": 1
          }
        ]
      }
    }
  ]
}
```

A.3 lib/CellHardwareControl/src/HardwareConfig.h

```
#ifndef HARDWARE_CONFIG_H
#define HARDWARE_CONFIG_H

#include <Arduino.h>

/**
 * @file HardwareConfig.h
 * @brief Pin definitions, hardware constants, and system configuration for
 *        Role 4: Cell Hardware & Alert Control (4-Cell Configuration).
 *
 * Designed for ESP32 DevKit V1 with zero pin conflicts against I2C (OLED & RTC) or UART.
 */

// ============================================================================
// SYSTEM CAPACITY & SCALABILITY CONFIGURATION (MATRIX GRIDS 2x2, 4x4, 8x8)
// ============================================================================

// Maximum system capacity for scalable matrix grids (up to 8x8 = 64 compartments)
#define HW_MAX_GRID_ROWS      8
```

```
#define HW_MAX_GRID_COLS      8
#define HW_MAX_CELLS          (HW_MAX_GRID_ROWS * HW_MAX_GRID_COLS) // 64 cells

#define HW_DIRECT_PIN_CELLS   4   // Direct GPIO wired cells on ESP32 DevKit (Cell 1 to 4)
#define DEFAULT_GRID_ROWS     2   // Default 2x2 grid layout
#define DEFAULT_GRID_COLS     2
#define DEFAULT_CELL_COUNT    (DEFAULT_GRID_ROWS * DEFAULT_GRID_COLS) // 4 cells

#ifndef MAX_CELLS
#define MAX_CELLS HW_MAX_CELLS
#endif

#ifndef ACTIVE_CELL_COUNT
#define ACTIVE_CELL_COUNT DEFAULT_CELL_COUNT
#endif

// ============================================================================
// PIN DEFINITIONS - I2C (RESERVED BY EXISTING MODULES - DO NOT ALTER)
// ============================================================================
// GPIO 21 -> I2C SDA (Used by SSD1306 OLED & DS1307 RTC)
// GPIO 22 -> I2C SCL (Used by SSD1306 OLED & DS1307 RTC)
// GPIO 1  -> UART TX0 (Used by Serial Monitor)
// GPIO 3  -> UART RX0 (Used by Serial Monitor)

// ============================================================================
// PIN DEFINITIONS - CELL 1 (COMPARTMENT 1)
// ============================================================================
#define PIN_CELL1_RED     25  // RGB Red channel (GPIO25)
#define PIN_CELL1_GREEN   26  // RGB Green channel (GPIO26)
#define PIN_CELL1_BLUE    -1  // Blue channel not needed (Red+Green = Yellow)
#define PIN_CELL1_BUTTON  32  // Push button (active LOW with internal pull-up)
#define PIN_CELL1_SERVO   18  // Compartment 1 Servo (PWM)

// ============================================================================
// PIN DEFINITIONS - CELL 2 (COMPARTMENT 2)
// ============================================================================
#define PIN_CELL2_RED     14  // RGB Red channel (GPIO14)
#define PIN_CELL2_GREEN   16  // RGB Green channel (GPIO16)
#define PIN_CELL2_BLUE    -1  // Blue channel not needed
#define PIN_CELL2_BUTTON  33  // Push button (active LOW with internal pull-up)
#define PIN_CELL2_SERVO   19  // Compartment 2 Servo (PWM)

// ============================================================================
// PIN DEFINITIONS - CELL 3 (COMPARTMENT 3)
// ============================================================================
#define PIN_CELL3_RED     13  // RGB Red channel (GPIO13)
#define PIN_CELL3_GREEN   12  // RGB Green channel (GPIO12)
#define PIN_CELL3_BLUE    -1  // Blue channel not needed
#define PIN_CELL3_BUTTON  27  // Push button (active LOW with internal pull-up)
#define PIN_CELL3_SERVO   5   // Compartment 3 Servo (PWM)

// ============================================================================
// PIN DEFINITIONS - CELL 4 (COMPARTMENT 4)
// ============================================================================
#define PIN_CELL4_RED     4   // RGB Red channel (GPIO4)
#define PIN_CELL4_GREEN   2   // RGB Green channel (GPIO2)
#define PIN_CELL4_BLUE    -1  // Blue channel not needed
#define PIN_CELL4_BUTTON  23  // Push button (active LOW with internal pull-up)
#define PIN_CELL4_SERVO   17  // Compartment 4 Servo (PWM)

// ============================================================================
// COMMON / SHARED ALERT ACTUATOR
// ============================================================================
#define PIN_COMMON_BUZZER    15  // Piezo Buzzer (PWM / Tone, GPIO15)
#define BUZZER_LEDC_CHANNEL  6   // Dedicated LEDC Channel 6 (Timer 3, no conflict with servos)

// ============================================================================
// TIMING CONSTANTS (NON-BLOCKING MILLIS)
// ============================================================================
#define DEBOUNCE_DELAY_MS           50    // Mechanical button debounce settling time (ms)
#define CONFIRM_GREEN_DURATION_MS   1000  // Duration to show GREEN after confirmation (ms)
#define BUZZER_FREQ_HZ              2500  // Resonant frequency for buzzer alert tone (Hz)
#define BUZZER_PULSE_ON_MS          300   // Buzzer intermittent pulse ON duration (ms)
```

```
#define BUZZER_PULSE_OFF_MS         300   // Buzzer intermittent pulse OFF duration (ms)

// ============================================================================
// SERVO CONFIGURATION
// ============================================================================
#define SERVO_CLOSED_ANGLE          0     // Closed / locked compartment position (degrees)
#define SERVO_OPEN_ANGLE            90    // Open / accessible compartment position (degrees)

// Structure for convenient per-cell pin bundling
struct CellPinConfig {
    int8_t pinRed;
    int8_t pinGreen;
    int8_t pinBlue;
    int8_t pinButton;
    int8_t pinServo;
};

// Default pin configurations for the direct physical cells
static const CellPinConfig DEFAULT_DIRECT_CELL_PIN_CONFIGS[HW_DIRECT_PIN_CELLS] = {
    { PIN_CELL1_RED, PIN_CELL1_GREEN, PIN_CELL1_BLUE, PIN_CELL1_BUTTON, PIN_CELL1_SERVO },
    { PIN_CELL2_RED, PIN_CELL2_GREEN, PIN_CELL2_BLUE, PIN_CELL2_BUTTON, PIN_CELL2_SERVO },
    { PIN_CELL3_RED, PIN_CELL3_GREEN, PIN_CELL3_BLUE, PIN_CELL3_BUTTON, PIN_CELL3_SERVO },
    { PIN_CELL4_RED, PIN_CELL4_GREEN, PIN_CELL4_BLUE, PIN_CELL4_BUTTON, PIN_CELL4_SERVO }
};

#define DEFAULT_CELL_PIN_CONFIGS DEFAULT_DIRECT_CELL_PIN_CONFIGS

#endif // HARDWARE_CONFIG_H
```

A.4 src/HardwareBridge.h

```
#ifndef HARDWARE_BRIDGE_H
#define HARDWARE_BRIDGE_H

#include <Arduino.h>

/**
 * @file HardwareBridge.h
 * @brief Integration bridge between Role 4 (Cell Hardware & Alert Control)
 *        and Role 2 (Scheduling & Event Engine) / Role 3 (OLED UI).
 *
 * Encapsulates the hardware controller and isolates header namespaces
 * to prevent enum collisions between CellState (hardware) and CellState (occupancy).
 */

// Function pointer for when physical button confirms a cell
typedef void (*BridgeConfirmCallback)(uint8_t cellIndex);

// Initialize cells (RGB LEDs, debounced buttons, servos) and common buzzer.
// Takes activeCellCount as input (defaults to 4).
void initHardwareBridge(BridgeConfirmCallback onConfirm, uint8_t activeCellCount = 4);

// Dynamically set the number of active cells at runtime (takes cell count as input)
bool setHardwareCellCount(uint8_t cellCount);

// Scalable Matrix Grid configuration (e.g. 2x2, 4x4, 8x8)
bool setHardwareGrid(uint8_t rows, uint8_t cols);
uint8_t getHardwareGridRows();
uint8_t getHardwareGridCols();
int8_t getHardwareCellIndex(uint8_t row, uint8_t col);
bool getHardwareCellGridCoords(uint8_t cellIndex, uint8_t& outRow, uint8_t& outCol);

// Periodic update loop (call continuously in loop())
void updateHardwareBridge();

// Inbound hardware event triggers (linear 0-indexed)
bool triggerHardwareMedicineDue(uint8_t cellIndex);
bool triggerHardwareMissedDose(uint8_t cellIndex);
bool triggerHardwareDoseConfirmed(uint8_t cellIndex);
bool triggerHardwareReset(uint8_t cellIndex);
void triggerHardwareResetAll();

// Inbound hardware event triggers (matrix 1-indexed: row, col)
```

```
bool triggerHardwareMedicineDueGrid(uint8_t row, uint8_t col);
bool triggerHardwareMissedDoseGrid(uint8_t row, uint8_t col);
bool triggerHardwareDoseConfirmedGrid(uint8_t row, uint8_t col);
bool triggerHardwareResetGrid(uint8_t row, uint8_t col);

// Queries
uint8_t getHardwareCellCount();
uint8_t getHardwareActiveAlertCount();
const char* getHardwareCellStateString(uint8_t cellIndex);
bool isHardwareCellServoOpen(uint8_t cellIndex);
uint8_t getHardwareCellServoAngle(uint8_t cellIndex);
bool isHardwareCellVirtual(uint8_t cellIndex);

// Simulate button press for virtual-expander cells (5+) via serial PRESS.
// Flows through update() -> scheduler callback, same as physical Btn 1-4.
bool pressHardwareButton(uint8_t cellIndex);
bool pressHardwareButtonGrid(uint8_t row, uint8_t col);

// Diagnostic status dump
void printHardwareStatus();

#endif // HARDWARE_BRIDGE_H
```

A.5 src/HardwareBridge.cpp

```
#include "HardwareBridge.h"
#include "CellHardwareEvents.h"

// Global instance of the 4-cell hardware controller (Role 4)
CellHardwareController cellHardware(ACTIVE_CELL_COUNT);

static BridgeConfirmCallback g_bridgeConfirmCallback = nullptr;
static bool g_suppressCallback = false;

/**
 * @brief Outbound Event Handler: DOSE_CONFIRMED
 *        Invoked when physical confirm button on a cell is debounced and pressed.
 */
static void internalConfirmCallback(uint8_t cellId) {
    if (g_suppressCallback) {
        return;
    }
    Serial.printf("\n>>> [HARDWARE EVENT] Physical DOSE_CONFIRMED button pressed on Cell %u!\n",
cellId + 1);
    Serial.printf("    Hardware Action: Cell %u RGB is GREEN for 1s. Servo %u is CLOSED.\n\n",
cellId + 1, cellId + 1);
    if (g_bridgeConfirmCallback) {
        g_bridgeConfirmCallback(cellId);
    }
}

/**
 * @brief State Change Observer Callback
 *        Invoked whenever any cell undergoes a state machine transition.
 */
static void internalStateChangedCallback(uint8_t cellId, CellState oldState, CellState newState) {
    Serial.printf("[STATE CHANGE] Cell %u: %s -> %s\n",
                  cellId + 1, cellStateToString(oldState), cellStateToString(newState));
}

void initHardwareBridge(BridgeConfirmCallback onConfirm, uint8_t activeCellCount) {
    g_bridgeConfirmCallback = onConfirm;
    cellHardware.setConfirmationCallback(internalConfirmCallback);
    cellHardware.setStateChangedCallback(internalStateChangedCallback);
    cellHardware.setCellCount(activeCellCount);
    cellHardware.begin();
}

bool setHardwareCellCount(uint8_t cellCount) {
    return cellHardware.setCellCount(cellCount);
}

bool setHardwareGrid(uint8_t rows, uint8_t cols) {
```

```
    return cellHardware.setGrid(rows, cols);
}

uint8_t getHardwareGridRows() {
    return cellHardware.getGridRows();
}

uint8_t getHardwareGridCols() {
    return cellHardware.getGridCols();
}

int8_t getHardwareCellIndex(uint8_t row, uint8_t col) {
    return cellHardware.gridCoordsToIndex(row, col);
}

bool getHardwareCellGridCoords(uint8_t cellIndex, uint8_t& outRow, uint8_t& outCol) {
    return cellHardware.indexToGridCoords(cellIndex, outRow, outCol);
}

void updateHardwareBridge() {
    cellHardware.update();
}

bool triggerHardwareMedicineDue(uint8_t cellIndex) {
    return cellHardware.triggerMedicineDue(cellIndex);
}

bool triggerHardwareMissedDose(uint8_t cellIndex) {
    return cellHardware.triggerMissedDose(cellIndex);
}

bool triggerHardwareDoseConfirmed(uint8_t cellIndex) {
    g_suppressCallback = true;
    bool res = cellHardware.triggerDoseConfirmed(cellIndex);
    g_suppressCallback = false;
    return res;
}

bool triggerHardwareReset(uint8_t cellIndex) {
    return cellHardware.triggerReset(cellIndex);
}

void triggerHardwareResetAll() {
    cellHardware.triggerResetAll();
}

bool triggerHardwareMedicineDueGrid(uint8_t row, uint8_t col) {
    return cellHardware.triggerMedicineDueGrid(row, col);
}

bool triggerHardwareMissedDoseGrid(uint8_t row, uint8_t col) {
    return cellHardware.triggerMissedDoseGrid(row, col);
}

bool triggerHardwareDoseConfirmedGrid(uint8_t row, uint8_t col) {
    g_suppressCallback = true;
    bool res = cellHardware.triggerDoseConfirmedGrid(row, col);
    g_suppressCallback = false;
    return res;
}

bool triggerHardwareResetGrid(uint8_t row, uint8_t col) {
    return cellHardware.triggerResetGrid(row, col);
}

uint8_t getHardwareCellCount() {
    return cellHardware.getCellCount();
}

uint8_t getHardwareActiveAlertCount() {
    return cellHardware.getActiveAlertCount();
}
```

```
const char* getHardwareCellStateString(uint8_t cellIndex) {
    return cellStateToString(cellHardware.getCellState(cellIndex));
}

bool isHardwareCellServoOpen(uint8_t cellIndex) {
    CellHardware* c = cellHardware.getCell(cellIndex);
    return (c && c->getServo().isOpen());
}

uint8_t getHardwareCellServoAngle(uint8_t cellIndex) {
    CellHardware* c = cellHardware.getCell(cellIndex);
    return (c ? c->getServo().getCurrentAngle() : 0);
}

bool isHardwareCellVirtual(uint8_t cellIndex) {
    return cellHardware.isCellVirtual(cellIndex);
}

bool pressHardwareButton(uint8_t cellIndex) {
    return cellHardware.injectVirtualButtonPress(cellIndex);
}

bool pressHardwareButtonGrid(uint8_t row, uint8_t col) {
    return cellHardware.injectVirtualButtonPressGrid(row, col);
}

void printHardwareStatus() {
    uint8_t rows = cellHardware.getGridRows();
    uint8_t cols = cellHardware.getGridCols();
    uint8_t total = cellHardware.getCellCount();

    Serial.println("\n=======================================================");
    Serial.printf(" HARDWARE STATUS: Matrix Grid [%ux%u] = %u Active Cells\n", rows, cols, total);
    Serial.printf(" Physical Direct GPIOs : Cells 1..%u\n",
                  HW_DIRECT_PIN_CELLS < total ? HW_DIRECT_PIN_CELLS : total);
    if (total > HW_DIRECT_PIN_CELLS) {
        Serial.printf(" Virtual/Expander GPIOs: Cells %u..%u\n", HW_DIRECT_PIN_CELLS + 1, total);
    }
    Serial.printf(" Active Alerts         : %u\n", cellHardware.getActiveAlertCount());
    Serial.printf(" Common Buzzer         : %s (Channel %u)\n",
                  cellHardware.getActiveAlertCount() > 0 ? "PULSING" : "SILENT",
                  BUZZER_LEDC_CHANNEL);
    Serial.println("-------------------------------------------------------");
    Serial.println(" Compartment Grid Matrix [Cell (Row,Col): State | Servo | Mode]:");

    for (uint8_t r = 1; r <= rows; r++) {
        Serial.printf("  Row %u: ", r);
        for (uint8_t c = 1; c <= cols; c++) {
            int8_t idx = cellHardware.gridCoordsToIndex(r, c);
            if (idx >= 0) {
                CellHardware* cell = cellHardware.getCell((uint8_t)idx);
                const char* st = cellHardware.isCellAlerting((uint8_t)idx) ? "DUE" :
                                 (cellHardware.isCellWarning((uint8_t)idx) ? "MISSED" :
                                 (cellHardware.isCellConfirmed((uint8_t)idx) ? "CONFIRM" : "IDLE"));
                uint8_t angle = (cell ? cell->getServo().getCurrentAngle() : 0);
                const char* mode = cellHardware.isCellVirtual((uint8_t)idx) ? "V" : "P";
                Serial.printf("[C%02u: %-7s %2u° %s] ", idx + 1, st, angle, mode);
            }
        }
        Serial.println();
    }
    Serial.println(" P=Physical GPIO (Btn/LED/Servo wired)  V=Virtual expander (PRESS <id> to
confirm)");
    Serial.println("=======================================================\n");
}
```

A.6 src/main.cpp

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <RTClib.h>
```

```
#include <string>
#include <ArduinoJson.h>

#include "HardwareBridge.h"
#include "schedule_event_engine.hpp"
#include "time_provider.hpp"
#include "oled_notification_engine.hpp"
#include "configuration_loader.hpp"
#include "configuration_adapter.hpp"
#include "dose_manager.h"

// Embedded configuration.json text file pointers from PlatformIO
extern const uint8_t configuration_json_start[]
asm("_binary_medicine_cell_data_config_configuration_json_start");
extern const uint8_t configuration_json_end[]
asm("_binary_medicine_cell_data_config_configuration_json_end");

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

// I2C Peripherals
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
RTC_DS1307 rtc;

// Role 2 & Role 3 Singletons
RTCTimeProvider rtcTimeProvider;
ScheduleEventEngine scheduleEngine(&rtcTimeProvider);
OledNotificationEngine oledUi;

// Role 5 Singleton: Dose & Inventory Stock Management
DoseManager doseManager(5 * 60);

// Forward declarations
void handleSerialCommands();

/**
 * @brief Scheduler Event Callback Handler
 *        Invoked whenever the Scheduling Engine emits an event
 *        (DOSE_DUE, DOSE_MISSED, DOSE_CONFIRMED, DOSE_DISMISSED, DAILY_RESET, LOW_STOCK_WARNING).
 *        Routes events to the 4-cell hardware controller (Role 4).
 *        Note: OLED UI (Role 3) receives these events independently via its own listener.
 */
void onScheduleEngineEvent(const DoseEvent& event, void* /*userData*/) {
    switch (event.type) {
        case EventType::DOSE_DUE:
            Serial.printf("\n>>> [SCHEDULER] DOSE_DUE: Cell %u (%s) - %s at %02u:%02u (Dose: %u)\n",
                          event.cellIndex + 1, event.cellId, event.medicineName,
                          event.scheduledTime.hour, event.scheduledTime.minute, event.doseQuantity);
            doseManager.addDose((uint16_t)(1000 + event.eventId), event.cellIndex + 1,
rtc.now().unixtime(), event.doseQuantity);
            doseManager.update(rtc.now().unixtime());
            // Trigger 4-Cell Hardware: LED RED, Servo OPEN to 90 deg, Buzzer pulsing
            triggerHardwareMedicineDue(event.cellIndex);
            break;

        case EventType::DOSE_MISSED:
            Serial.printf("\n>>> [SCHEDULER] DOSE_MISSED: Cell %u (%s) - %s at %02u:%02u (Grace
period expired)\n",
                          event.cellIndex + 1, event.cellId, event.medicineName,
                          event.scheduledTime.hour, event.scheduledTime.minute);
            doseManager.update(rtc.now().unixtime() + 3600);
            // Trigger 4-Cell Hardware: LED YELLOW (Warning), Servo CLOSED
            triggerHardwareMissedDose(event.cellIndex);
            break;

        case EventType::DOSE_CONFIRMED:
            Serial.printf("\n>>> [SCHEDULER] DOSE_CONFIRMED: Cell %u (%s) - %s (Dose %u taken)\n",
                          event.cellIndex + 1, event.cellId, event.medicineName,
event.doseQuantity);
            // Stock was decremented inside scheduleEngine.confirmDose; sync with DoseManager and
notify
            {
                EngineCell* cell = scheduleEngine.getCellMutable(event.cellIndex);
```

```
                uint16_t remStock = (cell) ? cell->currentStock : 0;
                uint16_t prevStock = remStock + event.doseQuantity;
                Serial.printf("[STOCK] Cell %u (%s) stock decremented: %u -> %u\n",
                              event.cellIndex + 1, event.cellId, prevStock, remStock);

                DoseResult result = doseManager.confirmCell(event.cellIndex + 1,
rtc.now().unixtime());
                if (result == DoseResult::NoDueDose) {
                    doseManager.decrementStock(event.cellIndex + 1, event.doseQuantity);
                }
                if (cell) {
                    doseManager.setCellStock(event.cellIndex + 1, remStock);
                }
                const CellInventory* inv = doseManager.getCell(event.cellIndex + 1);
                if (inv) {
                    const char* resStr = "OK";
                    if (inv->stock == 0) resStr = "OUT OF STOCK (0 units left)";
                    else if (inv->isLow()) resStr = "LOW STOCK WARNING";
                    Serial.printf("[DOSE_MANAGER] Cell %u stock: %u (Threshold: %u, Status: %s)\n",
                                  event.cellIndex + 1, inv->stock, inv->lowStockThreshold, resStr);
                }
                if (remStock == 0) {
                    Serial.printf("[STOCK ALERT] Cell %u is now OUT OF STOCK (0 units left)! Refill
needed.\n", event.cellIndex + 1);
                } else if (cell && remStock <= cell->lowStockThreshold) {
                    Serial.printf("[LOW STOCK] Cell %u is now LOW STOCK (%u units left)!\n",
event.cellIndex + 1, remStock);
                }
            }
            // Trigger 4-Cell Hardware: LED GREEN for 1s, Servo CLOSED
            triggerHardwareDoseConfirmed(event.cellIndex);
            break;

        case EventType::DOSE_DISMISSED:
            Serial.printf("\n>>> [SCHEDULER] DOSE_DISMISSED: Cell %u (%s) - %s\n",
                          event.cellIndex + 1, event.cellId, event.medicineName);
            doseManager.acknowledgeMissedAlerts();
            // Clear warning state for this cell -> LED OFF
            triggerHardwareReset(event.cellIndex);
            break;

        case EventType::DAILY_RESET:
            Serial.println("\n>>> [SCHEDULER] DAILY_RESET: Midnight rollover! Refreshing daily
schedules.");
            triggerHardwareResetAll();
            break;

        case EventType::LOW_STOCK_WARNING:
            if (event.doseQuantity == 0) {
                Serial.printf("\n>>> [STOCK ALERT] Cell %u (%s) - %s is OUT OF STOCK (0 units)!
Scheduled dose skipped. Please RESTOCK.\n",
                              event.cellIndex + 1, event.cellId, event.medicineName);
                triggerHardwareMissedDose(event.cellIndex); // Warning LED (yellow), servo stays
CLOSED
            } else {
                Serial.printf("\n>>> [SCHEDULER] LOW_STOCK_WARNING: Cell %u (%s) - %s (Remaining
Stock: %u)\n",
                              event.cellIndex + 1, event.cellId, event.medicineName,
event.doseQuantity);
            }
            break;

        default:
            break;
    }
}

/**
 * @brief Outbound Event Handler: Physical Button Press on a Cell
 *        Invoked by the debounced button logic when a patient presses the button on a cell.
 *        Passes the confirmation to the scheduler, which confirms active DUE doses
 *        or dismisses active MISSED dose notifications, updating the OLED and hardware.
 */
```

```
void onHardwareDoseConfirmed(uint8_t cellId) {
    uint8_t cellNum = cellId + 1;
    Serial.printf("[CONFIRM ACTION] Button pressed for Cell %u. Checking stock and scheduler...\n",
cellNum);

    EngineCell* cell = scheduleEngine.getCellMutable(cellId);
    const CellInventory* inv = doseManager.getCell(cellNum);
    uint16_t currentStock = (cell) ? cell->currentStock : (inv ? inv->stock : 0);

    if (currentStock == 0) {
        Serial.printf("[STOCK ALERT] Cell %u button pressed, but stock is 0 (OUT OF STOCK)! Cannot
dispense.\n", cellNum);
        triggerHardwareReset(cellId);
        return;
    }

    bool handled = scheduleEngine.handleCellButtonPress(cellId);
    if (!handled) {
        // If not handled by scheduler, check if doseManager has an active due dose
        DoseResult res = doseManager.confirmCell(cellNum, rtc.now().unixtime());
        if (res == DoseResult::Confirmed || res == DoseResult::ConfirmedLowStock || res ==
DoseResult::ConfirmedStockEmpty) {
            uint16_t rem = doseManager.getCell(cellNum)->stock;
            if (cell) {
                cell->currentStock = rem;
            }
            Serial.printf("[DOSE_MANAGER] Cell %u confirmed via DoseManager! Remaining stock: %u\n",
                          cellNum, rem);
            triggerHardwareDoseConfirmed(cellId);
        } else {
            Serial.printf("[CONFIRM] Cell %u has no active due dose to confirm (Stock unchanged: %u
units).\n",
                          cellNum, currentStock);
            triggerHardwareReset(cellId);
        }
    }
}

void setup() {
    Serial.begin(115200);

    // 1. Initialize I2C Bus (D21 SDA, D22 SCL preserved)
    Wire.begin(21, 22);

    // 2. Initialize Hardware RTC
    if (!rtc.begin()) {
        Serial.println("RTC initialization failed!");
        while (true);
    }
    if (!rtc.isrunning()) {
        Serial.println("RTC was not running, setting to compile time...");
        rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
    }
    Serial.println("RTC initialized!");

    // 3. Load dynamic configuration from embedded configuration.json (No hardcoding!)
    size_t configLen = configuration_json_end - configuration_json_start;
    std::string configJson(reinterpret_cast<const char*>(configuration_json_start), configLen);

    RuntimeConfiguration runtimeConfig;
    uint8_t targetCellCount = 4;
    if (loadConfiguration(configJson.c_str(), runtimeConfig)) {
        Serial.printf("[CONFIG] Successfully loaded configuration.json (%u cells configured):\n",
runtimeConfig.cellCount);
        for (uint8_t i = 0; i < runtimeConfig.cellCount; i++) {
            RuntimeCell& c = runtimeConfig.cells[i];
            Serial.printf("  Cell %u [%s]: %s (Stock: %u)\n",
                          i + 1, c.cellId, c.hasMedicine ? c.medicine.name : "EMPTY",
c.currentStock);
            doseManager.addCell(i + 1, c.hasMedicine ? c.medicine.name : "Cell", c.currentStock,
c.medicine.lowStockThreshold);
            DateTime now = rtc.now();
            for (uint8_t s = 0; s < c.medicine.scheduleCount; s++) {
```

```
                uint8_t sh = c.medicine.schedules[s].hour;
                uint8_t sm = c.medicine.schedules[s].minute;
                uint16_t sq = c.medicine.schedules[s].doseQuantity;
                Serial.printf("    - Schedule: %02u:%02u (Dose: %u)\n", sh, sm, sq);
                DateTime schedDt(now.year(), now.month(), now.day(), sh, sm, 0);
                uint16_t doseId = (uint16_t)((i + 1) * 100 + s + 1);
                doseManager.addDose(doseId, i + 1, schedDt.unixtime(), sq);
            }
        }
        doseManager.update(rtc.now().unixtime());
        if (runtimeConfig.cellCount > 0 && runtimeConfig.cellCount <= MAX_CELLS) {
            targetCellCount = runtimeConfig.cellCount;
        }
    } else {
        Serial.println("[CONFIG] ERROR: Failed to parse configuration.json! Defaulting to 4
cells.");
    }

    Serial.printf("[INIT] Starting system with %u active cell(s).\n\n", targetCellCount);

    // 4. Initialize Role 4: Cell Hardware & Alert Control with dynamic cell count
    initHardwareBridge(onHardwareDoseConfirmed, targetCellCount);

    // Check if configuration.json specifies an explicit hardware grid (e.g. "4x4", "8x8")
    // If "auto", derive grid from cellCount with the same mapping as scripts/set_diagram_grid.py
    // so firmware + Wokwi canvas stay in sync automatically.
    JsonDocument cfgDoc;
    if (!deserializeJson(cfgDoc, configJson.c_str())) {
        const char* explicitGrid = cfgDoc["grid"] | "auto";
        if (explicitGrid && strcmp(explicitGrid, "auto") != 0) {
            int gr = 0, gc = 0;
            if (sscanf(explicitGrid, "%dx%d", &gr, &gc) == 2 || sscanf(explicitGrid, "%dX%d", &gr,
&gc) == 2) {
                if (gr >= 1 && gr <= 8 && gc >= 1 && gc <= 8 && (gr * gc) <= 64) {
                    setHardwareGrid((uint8_t)gr, (uint8_t)gc);
                    Serial.printf("[CONFIG] Explicit hardware matrix configured to [%ux%u] = %u
compartments!\n",
                                  gr, gc, gr * gc);
                    if ((uint8_t)(gr * gc) != targetCellCount) {
                        Serial.printf("[CONFIG] NOTE: grid %ux%u (%u) overrides cellCount %u. "
                                      "Set both to match (e.g. 4/2x2, 16/4x4) for full matrix.\n",
                                      gr, gc, gr * gc, targetCellCount);
                    }
                }
            }
        } else {
            // auto: 1->1x1, 2->1x2, <=4->2x2, <=16->4x4, else 8x8 (matches diagram generator)
            uint8_t ar = 1, ac = targetCellCount;
            if (targetCellCount <= 1) { ar = 1; ac = 1; }
            else if (targetCellCount <= 2) { ar = 1; ac = 2; }
            else if (targetCellCount <= 4) { ar = 2; ac = 2; }
            else if (targetCellCount <= 16) { ar = 4; ac = 4; }
            else { ar = 8; ac = 8; }
            setHardwareGrid(ar, ac);
            Serial.printf("[CONFIG] Auto grid derived from cellCount %u -> [%ux%u].\n",
                          targetCellCount, ar, ac);
        }
        size_t cellsInJson = cfgDoc["cells"].is<JsonArray>() ?
cfgDoc["cells"].as<JsonArray>().size() : 0;
        if (cellsInJson != getHardwareCellCount()) {
            Serial.printf("[CONFIG] WARNING: cells[] has %u entries but hardware has %u active. "
                          "Align cellCount/grid/cells[] in configuration.json.\n",
                          (unsigned)cellsInJson, getHardwareCellCount());
        }
    }

    Serial.printf("Role 4: Cell Hardware & Alert Control initialized with %u active cells (Grid
[%ux%u])!\n",
                  getHardwareCellCount(), getHardwareGridRows(), getHardwareGridCols());
    if (getHardwareCellCount() > 4) {
        Serial.println("[CONFIG] Cells 1-4 = physical GPIO; Cells 5+ = virtual expander (LED/servo
logged, PRESS <id> to confirm).");
    }
```

```
    // 5. Initialize Role 2: Scheduling & Time/Event Engine
    scheduleEngine.setTimeProvider(&rtcTimeProvider);
    scheduleEngine.setGracePeriodMinutes(5); // 5-minute grace window for alerts before marking
MISSED
    scheduleEngine.registerEventCallback(onScheduleEngineEvent);
    if (runtimeConfig.cellCount > 0) {
        ConfigurationAdapter::loadIntoEngine(runtimeConfig, scheduleEngine);
        Serial.println("[CONFIG] Schedules dynamically loaded into Scheduling Engine!");
    }

    // 6. Initialize Role 3: OLED UI & Notification Engine
    if (!oledUi.begin(display, scheduleEngine)) {
        Serial.println("OLED UI initialization failed!");
        while (true);
    }
    char initGridLabel[16];
    snprintf(initGridLabel, sizeof(initGridLabel), "[%ux%u]", getHardwareGridRows(),
getHardwareGridCols());
    oledUi.setGridLabel(initGridLabel);
    Serial.println("Role 3: OLED UI & Notification Engine initialized!");

    Serial.println("================================================================");
    Serial.println(" Smart Medicine Reminder System - Scalable Matrix Hardware Active");
    Serial.printf(" Matrix Grid: [%ux%u] | Cells: %u | Common Buzzer: ON\n",
                  getHardwareGridRows(), getHardwareGridCols(), getHardwareCellCount());
    Serial.println("================================================================");
    Serial.println("Serial Commands:");
    Serial.println("  STATUS              - Print full visual matrix hardware & event status");
    Serial.println("  GRID <RxC>          - Scale matrix grid (e.g. GRID 2x2, GRID 4x4, GRID 8x8)");
    Serial.println("  CELLS <1-64>        - Dynamically set number of active cells");
    Serial.println("  CONFIRM <id | r,c>  - Confirm dose for Cell (e.g. CONFIRM 1 or CONFIRM 2,1)");
    Serial.println("  PRESS <id | r,c>    - Simulate button press (required for virtual Cells 5+)");
    Serial.println("  DUE <id | r,c>      - Manually trigger MEDICINE_DUE for Cell (or row,col)");
    Serial.println("  MISSED <id | r,c>   - Manually trigger MISSED_DOSE for Cell (or row,col)");
    Serial.println("  RESET <id | r,c>    - Reset Cell (or row,col) to IDLE");
    Serial.println("  RESET               - Reset all active cells & dismiss all reminders");
    Serial.println("  LOWSTOCK <1-4>      - Simulate low stock for Cell X (OLED alert)");
    Serial.println("  RESTOCK <1-N> <qty> - Refill medicine stock using DoseManager");
    Serial.println("  TIME <HH> <MM> [SS] - Set RTC time (e.g., TIME 07 59 55)");
    Serial.println("  GRACE <minutes>     - Set missed dose grace period in minutes");
    Serial.println("Or click buttons 'Btn 1' - 'Btn 4' on virtual board! For Cells 5+, use PRESS
<id>.");
    Serial.println("================================================================");
}

// Periodic heartbeat timer
static uint32_t lastSerialHeartbeat = 0;

void loop() {
    // ------------------------------------------------------------
    // 1. Role 4: Cell Hardware & Alert Control (Run continuously)
    //    Debounces buttons, manages RGB LEDs, servos, buzzer cadence.
    // ------------------------------------------------------------
    updateHardwareBridge();

    // ------------------------------------------------------------
    // 2. Role 2: Scheduling & Time/Event Engine
    //    Polls RTC time, detects due schedules, tracks grace periods,
    //    handles simultaneous due events and midnight rollovers.
    // ------------------------------------------------------------
    scheduleEngine.update();

    // ------------------------------------------------------------
    // 3. Role 3: OLED UI & Notification Engine
    //    Non-blocking: renders clock screen, active due notifications,
    //    missed dose warnings, confirmation confirmations, rotating multi-doses.
    // ------------------------------------------------------------
    oledUi.update();

    // ------------------------------------------------------------
    // 4. Role 5: Dose, Stock & Confirmation Manager
    // ------------------------------------------------------------
```

```
    doseManager.update(rtc.now().unixtime());

    // ------------------------------------------------------------
    // 5. Interactive Serial Command Processing
    // ------------------------------------------------------------
    handleSerialCommands();

    // ------------------------------------------------------------
    // 5. Periodic Heartbeat Log to Serial Monitor (Every 5 seconds)
    // ------------------------------------------------------------
    uint32_t currentMillis = millis();
    if (currentMillis - lastSerialHeartbeat >= 5000) {
        lastSerialHeartbeat = currentMillis;

        DateTime now = rtc.now();
        Serial.printf("[HEARTBEAT] RTC: %02u:%02u:%02u | Due: %u | Missed: %u | Alerting Cells: %u |
Buzzer: %s\n",
                      now.hour(), now.minute(), now.second(),
                      scheduleEngine.getDueEventCount(),
                      scheduleEngine.getMissedEventCount(),
                      getHardwareActiveAlertCount(),
                      getHardwareActiveAlertCount() > 0 ? "PULSING" : "SILENT");
    }
}

/**
 * @brief Parse interactive test commands from Serial input.
 */
void handleSerialCommands() {
    if (!Serial.available()) return;

    String line = Serial.readStringUntil('\n');
    line.trim();

    if (line.length() == 0) return;

    String upperLine = line;
    upperLine.toUpperCase();

    if (upperLine == "STATUS") {
        printHardwareStatus();

        DateTime now = rtc.now();
        Serial.printf("Current RTC Time    : %02u:%02u:%02u\n", now.hour(), now.minute(),
now.second());
        Serial.printf("Grace Period        : %u minutes\n", scheduleEngine.getGracePeriodMinutes());
        Serial.printf("Active Due Events   : %u\n", scheduleEngine.getDueEventCount());
        for (uint8_t i = 0; i < scheduleEngine.getDueEventCount(); i++) {
            const DoseEvent* ev = scheduleEngine.getDueEvent(i);
            if (ev) {
                Serial.printf("  - DUE: Cell %u (%s), %s, Sched: %02u:%02u, Qty: %u\n",
                              ev->cellIndex + 1, ev->cellId, ev->medicineName,
                              ev->scheduledTime.hour, ev->scheduledTime.minute, ev->doseQuantity);
            }
        }
        Serial.printf("Active Missed Events: %u\n", scheduleEngine.getMissedEventCount());
        for (uint8_t i = 0; i < scheduleEngine.getMissedEventCount(); i++) {
            const DoseEvent* ev = scheduleEngine.getMissedEvent(i);
            if (ev) {
                Serial.printf("  - MISSED: Cell %u (%s), %s, Sched: %02u:%02u\n",
                              ev->cellIndex + 1, ev->cellId, ev->medicineName,
                              ev->scheduledTime.hour, ev->scheduledTime.minute);
            }
        }
        Serial.println("Inventory Stock Status (DoseManager + Scheduler):");
        for (uint8_t i = 1; i <= getHardwareCellCount(); i++) {
            const CellInventory* inv = doseManager.getCell(i);
            const EngineCell* cell = scheduleEngine.getCell(i - 1);
            uint16_t stock = (cell) ? cell->currentStock : (inv ? inv->stock : 0);
            uint16_t threshold = (inv) ? inv->lowStockThreshold : (cell ? cell->lowStockThreshold :
0);
            const char* statusStr = "OK";
            if (stock == 0) statusStr = "OUT OF STOCK [0 LEFT]";
```

```
            else if (stock <= threshold) statusStr = "LOW STOCK WARNING";
            Serial.printf("  - Cell %u (%s): Stock %u | Threshold %u | Status: %s\n",
                          i, inv ? inv->medicineName.c_str() : (cell ? cell->medicineName :
"EMPTY"),
                          stock, threshold, statusStr);
        }
        if (doseManager.hasMissedAlert()) {
            Serial.printf("  [ALERT] DoseManager has pending missed dose alerts (Count: %u)\n",
                          (unsigned)doseManager.missedDoseCount());
        }
        Serial.println("----------------------------------------\n");
        return;
    }

    if (upperLine.startsWith("TIME ")) {
        // Syntax: TIME <HH> <MM> [SS]
        String args = line.substring(5);
        args.trim();
        int h = 0, m = 0, s = 0;
        int firstSpace = args.indexOf(' ');
        if (firstSpace > 0) {
            h = args.substring(0, firstSpace).toInt();
            String rest = args.substring(firstSpace + 1);
            rest.trim();
            int secondSpace = rest.indexOf(' ');
            if (secondSpace > 0) {
                m = rest.substring(0, secondSpace).toInt();
                s = rest.substring(secondSpace + 1).toInt();
            } else {
                m = rest.toInt();
                s = 0;
            }
            if (h >= 0 && h < 24 && m >= 0 && m < 60 && s >= 0 && s < 60) {
                rtc.adjust(DateTime(2026, 9, 28, h, m, s));
                oledUi.requestRedraw();
                Serial.printf("[COMMAND] Set RTC Time to %02d:%02d:%02d\n", h, m, s);
                return;
            }
        }
        Serial.println("Invalid format. Use: TIME <HH> <MM> [SS] (e.g., TIME 07 59 55)");
        return;
    }

    auto parseCellArg = [](const String& arg) -> int {
        int commaIdx = arg.indexOf(',');
        if (commaIdx > 0) {
            int r = arg.substring(0, commaIdx).toInt();
            int c = arg.substring(commaIdx + 1).toInt();
            int8_t idx = getHardwareCellIndex((uint8_t)r, (uint8_t)c);
            if (idx >= 0) return idx + 1; // return 1-indexed cell number
            return -1;
        }
        return arg.toInt();
    };

    if (upperLine.startsWith("GRID ")) {
        String args = line.substring(5);
        args.trim();
        int r = 0, c = 0;
        int xPos = args.indexOf('x');
        if (xPos < 0) xPos = args.indexOf('X');
        int spacePos = args.indexOf(' ');

        if (xPos > 0) {
            r = args.substring(0, xPos).toInt();
            c = args.substring(xPos + 1).toInt();
        } else if (spacePos > 0) {
            r = args.substring(0, spacePos).toInt();
            c = args.substring(spacePos + 1).toInt();
        }

        if (r >= 1 && r <= 8 && c >= 1 && c <= 8 && (r * c) <= 64) {
            bool ok = setHardwareGrid((uint8_t)r, (uint8_t)c);
```

```
            if (ok) {
                Serial.printf("[COMMAND] Hardware grid scaled to [%ux%u] = %u active cells!\n", r,
c, r * c);
                printHardwareStatus();

                char gridStr[16];
                snprintf(gridStr, sizeof(gridStr), "[%ux%u]", r, c);
                oledUi.setGridLabel(gridStr);
            } else {
                Serial.printf("[COMMAND] Failed to set hardware grid [%ux%u].\n", r, c);
            }
        } else {
            Serial.println("Invalid grid dimensions. Max: 8x8 (64 cells). E.g.: GRID 2 2, GRID 4 4,
GRID 8 8, GRID 4x4");
        }
        return;
    }

    if (upperLine.startsWith("CELLS ") || upperLine.startsWith("SET_CELLS ")) {
        int spaceIdx = upperLine.indexOf(' ');
        int count = line.substring(spaceIdx + 1).toInt();
        if (count >= 1 && count <= 64) {
            bool ok = setHardwareCellCount((uint8_t)count);
            if (ok) {
                Serial.printf("[COMMAND] Dynamic active cell count updated to %d cell(s) (Grid
[%ux%u])!\n",
                              count, getHardwareGridRows(), getHardwareGridCols());
                printHardwareStatus();

                char gridStr[16];
                snprintf(gridStr, sizeof(gridStr), "[%ux%u]", getHardwareGridRows(),
getHardwareGridCols());
                oledUi.setGridLabel(gridStr);
            } else {
                Serial.printf("[COMMAND] Failed to set cell count to %d.\n", count);
            }
        } else {
            Serial.println("Invalid cell count. Range: 1 to 64 cells (e.g., CELLS 16).");
        }
        return;
    }

    if (upperLine.startsWith("GRACE ")) {
        int minutes = line.substring(6).toInt();
        if (minutes > 0 && minutes <= 120) {
            scheduleEngine.setGracePeriodMinutes(minutes);
            Serial.printf("[COMMAND] Grace period set to %d minutes\n", minutes);
        } else {
            Serial.println("Invalid grace period. Range: 1 to 120 minutes.");
        }
        return;
    }

    if (upperLine.startsWith("PRESS ") || upperLine.startsWith("BTN ") ||
upperLine.startsWith("BUTTON ")) {
        int spaceIdx = upperLine.indexOf(' ');
        String arg = line.substring(spaceIdx + 1);
        arg.trim();
        int cellNum = parseCellArg(arg);
        if (cellNum >= 1 && cellNum <= (int)getHardwareCellCount()) {
            uint8_t r = 0, c = 0;
            getHardwareCellGridCoords(cellNum - 1, r, c);
            Serial.printf("[COMMAND] Simulating button PRESS for Cell %d (Row %u, Col %u)%s\n",
                          cellNum, r, c,
                          isHardwareCellVirtual(cellNum - 1) ? " [VIRTUAL expander]" : "
[PHYSICAL]");
            if (!pressHardwareButton(cellNum - 1)) {
                Serial.printf("[COMMAND] Failed to inject press for Cell %d.\n", cellNum);
            } else {
                Serial.println("[HINT] Press flows through update() -> scheduler; check STATUS /
OLED / buzzer.");
            }
        } else {
```

```
            Serial.printf("Invalid cell: '%s' (Available: 1 to %u, or Row,Col up to %u,%u)\n",
                          arg.c_str(), getHardwareCellCount(), getHardwareGridRows(),
getHardwareGridCols());
        }
        return;
    }

    if (upperLine.startsWith("DUE ")) {
        String arg = line.substring(4);
        arg.trim();
        int cellNum = parseCellArg(arg);
        if (cellNum >= 1 && cellNum <= (int)getHardwareCellCount()) {
            uint8_t r = 0, c = 0;
            getHardwareCellGridCoords(cellNum - 1, r, c);

            EngineCell* cell = scheduleEngine.getCellMutable(cellNum - 1);
            const CellInventory* inv = doseManager.getCell(cellNum);
            uint16_t currentStock = (cell) ? cell->currentStock : (inv ? inv->stock : 0);

            if (currentStock == 0) {
                Serial.printf("[STOCK ALERT] Cannot trigger DUE for Cell %d (Row %u, Col %u): OUT OF
STOCK (Stock: 0)!\n",
                              cellNum, r, c);
                Serial.printf("[STOCK ALERT] Servo kept CLOSED. Please use 'RESTOCK %d <qty>'
first.\n", cellNum);
                triggerHardwareMissedDose(cellNum - 1); // Yellow warning LED, servo stays CLOSED
                oledUi.requestRedraw();
                return;
            }

            Serial.printf("[COMMAND] Triggering MEDICINE_DUE for Cell %d (Row %u, Col %u) - Stock:
%u\n",
                          cellNum, r, c, currentStock);
            bool ok = scheduleEngine.triggerDueEvent(cellNum - 1);
            if (!ok) {
                uint16_t reqQty = 1;
                if (cell && cell->scheduleCount > 0) {
                    reqQty = cell->schedules[0].doseQuantity;
                }
                doseManager.addDose((uint16_t)(1000 + cellNum), cellNum, rtc.now().unixtime(),
reqQty);
                doseManager.update(rtc.now().unixtime());
            }
        } else {
            Serial.printf("Invalid cell: '%s' (Available: 1 to %u, or Row,Col up to %u,%u)\n",
                          arg.c_str(), getHardwareCellCount(), getHardwareGridRows(),
getHardwareGridCols());
        }
        return;
    } else if (upperLine.startsWith("MISSED ")) {
        String arg = line.substring(7);
        arg.trim();
        int cellNum = parseCellArg(arg);
        if (cellNum >= 1 && cellNum <= (int)getHardwareCellCount()) {
            uint8_t r = 0, c = 0;
            getHardwareCellGridCoords(cellNum - 1, r, c);
            Serial.printf("[COMMAND] Triggering MISSED_DOSE for Cell %d (Row %u, Col %u)\n",
cellNum, r, c);
            triggerHardwareMissedDose(cellNum - 1);
        } else {
            Serial.printf("Invalid cell: '%s' (Available: 1 to %u, or Row,Col up to %u,%u)\n",
                          arg.c_str(), getHardwareCellCount(), getHardwareGridRows(),
getHardwareGridCols());
        }
    } else if (upperLine.startsWith("CONFIRM ")) {
        String arg = line.substring(8);
        arg.trim();
        int cellNum = parseCellArg(arg);
        if (cellNum >= 1 && cellNum <= (int)getHardwareCellCount()) {
            uint8_t r = 0, c = 0;
            getHardwareCellGridCoords(cellNum - 1, r, c);

            EngineCell* cell = scheduleEngine.getCellMutable(cellNum - 1);
```

```
            const CellInventory* inv = doseManager.getCell(cellNum);
            uint16_t currentStock = (cell) ? cell->currentStock : (inv ? inv->stock : 0);

            if (currentStock == 0) {
                Serial.printf("[STOCK ERROR] Cannot confirm Cell %d (Row %u, Col %u): Stock is 0
(OUT OF STOCK)!\n",
                              cellNum, r, c);
                Serial.printf("[STOCK ERROR] Dose cannot be taken from an empty compartment. Refill
with 'RESTOCK %d <qty>'.\n",
                              cellNum);
                return;
            }

            Serial.printf("[COMMAND] Confirming dose for Cell %d (Row %u, Col %u)\n", cellNum, r,
c);
            bool handled = scheduleEngine.handleCellButtonPress(cellNum - 1);
            if (!handled) {
                // Check if doseManager has an active due occurrence
                DoseResult res = doseManager.confirmCell((uint8_t)cellNum, rtc.now().unixtime());
                if (res == DoseResult::Confirmed || res == DoseResult::ConfirmedLowStock || res ==
DoseResult::ConfirmedStockEmpty) {
                    uint16_t rem = doseManager.getCell((uint8_t)cellNum)->stock;
                    if (cell) {
                        cell->currentStock = rem;
                    }
                    Serial.printf("[DOSE_MANAGER] Cell %d confirmed via DoseManager! Remaining
stock: %u\n", cellNum, rem);
                    triggerHardwareDoseConfirmed(cellNum - 1);
                } else {
                    Serial.printf("[COMMAND] Cell %d has no active due dose to confirm (Stock
unchanged: %u units).\n",
                                  cellNum, currentStock);
                }
            }
        } else {
            Serial.printf("Invalid cell: '%s' (Available: 1 to %u, or Row,Col up to %u,%u)\n",
                          arg.c_str(), getHardwareCellCount(), getHardwareGridRows(),
getHardwareGridCols());
        }
        return;
    } else if (upperLine.startsWith("RESET ")) {
        String arg = line.substring(6);
        arg.trim();
        int cellNum = parseCellArg(arg);
        if (cellNum >= 1 && cellNum <= (int)getHardwareCellCount()) {
            uint8_t r = 0, c = 0;
            getHardwareCellGridCoords(cellNum - 1, r, c);
            Serial.printf("[COMMAND] Resetting Cell %d (Row %u, Col %u) to IDLE\n", cellNum, r, c);
            triggerHardwareReset(cellNum - 1);
            scheduleEngine.dismissMissedDose(cellNum - 1);
        } else {
            Serial.printf("Invalid cell: '%s' (Available: 1 to %u, or Row,Col up to %u,%u)\n",
                          arg.c_str(), getHardwareCellCount(), getHardwareGridRows(),
getHardwareGridCols());
        }
    } else if (upperLine == "RESET") {
        Serial.printf("[COMMAND] Resetting all %u cells to IDLE & dismissing missed reminders\n",
getHardwareCellCount());
        triggerHardwareResetAll();
        scheduleEngine.dismissAllMissedDoses();
    } else if (upperLine.startsWith("LOWSTOCK ")) {
        // Manually set a cell's stock to 1 to trigger a LOW_STOCK_WARNING on next engine tick.
        int cellNum = line.substring(9).toInt();
        if (cellNum >= 1 && cellNum <= (int)scheduleEngine.getCellCount()) {
            EngineCell* cell = scheduleEngine.getCellMutable(cellNum - 1);
            if (cell) {
                cell->currentStock = 1;  // Force stock below any threshold
                doseManager.setCellStock((uint8_t)cellNum, 1);
                Serial.printf("[COMMAND] Cell %d stock forced to 1 (LOW_STOCK_WARNING expected on
next update)\n", cellNum);
            }
        } else {
```

```
            Serial.printf("Invalid cell number: %d (Available: 1 to %u)\n", cellNum,
scheduleEngine.getCellCount());
        }
    } else if (upperLine.startsWith("EMPTY ") || upperLine.startsWith("ZEROSTOCK ")) {
        int spaceIdx = upperLine.indexOf(' ');
        int cellNum = line.substring(spaceIdx + 1).toInt();
        if (cellNum >= 1 && cellNum <= (int)scheduleEngine.getCellCount()) {
            EngineCell* cell = scheduleEngine.getCellMutable(cellNum - 1);
            if (cell) {
                cell->currentStock = 0;
                doseManager.setCellStock((uint8_t)cellNum, 0);
                Serial.printf("[COMMAND] Cell %d stock forced to 0 (OUT OF STOCK)\n", cellNum);
                triggerHardwareMissedDose(cellNum - 1); // Yellow warning LED, servo CLOSED
            }
        } else {
            Serial.printf("Invalid cell number: %d (Available: 1 to %u)\n", cellNum,
scheduleEngine.getCellCount());
        }
    } else if (upperLine.startsWith("RESTOCK ")) {
        int cellNum = 0, units = 0;
        if (sscanf(line.c_str() + 8, "%d %d", &cellNum, &units) == 2 && cellNum >= 1 && units > 0) {
            bool ok = doseManager.restockCell((uint8_t)cellNum, (uint16_t)units);
            if (cellNum <= (int)scheduleEngine.getCellCount()) {
                EngineCell* cell = scheduleEngine.getCellMutable(cellNum - 1);
                if (cell) {
                    cell->currentStock += units;
                    doseManager.setCellStock((uint8_t)cellNum, cell->currentStock);
                }
            }
            if (ok) {
                const CellInventory* inv = doseManager.getCell((uint8_t)cellNum);
                uint16_t s = inv ? inv->stock : 0;
                Serial.printf("[COMMAND] Cell %d restocked +%d units. New stock: %u (Status: %s)\n",
                              cellNum, units, s,
                              (s == 0) ? "OUT OF STOCK" : ((inv && inv->isLow()) ? "LOW STOCK" :
"OK"));
                triggerHardwareReset(cellNum - 1);
            } else {
                Serial.printf("[COMMAND] Failed to restock Cell %d (invalid cell or overflow)\n",
cellNum);
            }
        } else {
            Serial.println("Usage: RESTOCK <cellNum> <units> (e.g. RESTOCK 1 10)");
        }
    } else {
        Serial.printf("Unknown command: '%s'. Try: STATUS, GRID, CELLS, DUE, CONFIRM, PRESS, MISSED,
RESET, LOWSTOCK, EMPTY, RESTOCK, TIME, GRACE\n", line.c_str());
    }
}
```

A.7 include/dose_manager.h

```
#ifndef DOSE_MANAGER_H
#define DOSE_MANAGER_H

#include <stdint.h>
#include <stddef.h>
#include <string>
#include <vector>

enum class DoseStatus : uint8_t {
    Scheduled,
    Due,
    Confirmed,
    Missed
};

enum class DoseResult : uint8_t {
    Confirmed,
    ConfirmedLowStock,
    ConfirmedStockEmpty,
    NoDueDose,
    MissedDoseNotCounted,
```

```
    UnknownCell
};

struct CellInventory {
    uint8_t cellId;
    std::string medicineName;
    uint16_t stock;
    uint16_t lowStockThreshold;

    bool isLow() const {
        return stock <= lowStockThreshold;
    }
};

struct DoseOccurrence {
    uint16_t doseId;
    uint8_t cellId;
    uint64_t scheduledAt;
    uint16_t units;
    DoseStatus status;
    bool missedAlertPending;
};

// Tracks scheduled dose occurrences and the stock held by each medicine cell.
// The caller supplies Unix timestamps from the RTC and persists data separately
// if state must survive a power loss.
class DoseManager {
public:
    explicit DoseManager(uint32_t missedGraceSeconds = 15UL * 60UL)
        : missedGraceSeconds_(missedGraceSeconds), missedAlertPending_(false) {}

    bool addCell(uint8_t cellId, const std::string& medicineName,
                 uint16_t initialStock, uint16_t lowStockThreshold) {
        if (findCell(cellId) != NULL) {
            return false;
        }

        CellInventory cell = {cellId, medicineName, initialStock, lowStockThreshold};
        cells_.push_back(cell);
        return true;
    }

    bool addDose(uint16_t doseId, uint8_t cellId, uint64_t scheduledAt,
                 uint16_t units = 1) {
        if (findCell(cellId) == NULL || units == 0 || findDose(doseId) != NULL) {
            return false;
        }

        DoseOccurrence dose = {
            doseId, cellId, scheduledAt, units, DoseStatus::Scheduled, false
        };
        doses_.push_back(dose);
        return true;
    }

    // Add stock for a refill. Reject invalid cells, zero refills, and overflow.
    bool restockCell(uint8_t cellId, uint16_t units) {
        CellInventory* cell = findCell(cellId);
        if (cell == NULL || units == 0 ||
            units > static_cast<uint16_t>(UINT16_MAX - cell->stock)) {
            return false;
        }
        cell->stock = static_cast<uint16_t>(cell->stock + units);
        return true;
    }

    // Directly decrement stock by units. Clamps at zero.
    bool decrementStock(uint8_t cellId, uint16_t units = 1) {
        CellInventory* cell = findCell(cellId);
        if (cell == NULL || cell->stock == 0) {
            return false;
        }
        if (cell->stock < units) {
```

```
        cell->stock = 0;
    } else {
        cell->stock = static_cast<uint16_t>(cell->stock - units);
    }
    return true;
}

bool setCellStock(uint8_t cellId, uint16_t newStock) {
    CellInventory* cell = findCell(cellId);
    if (cell == NULL) {
        return false;
    }
    cell->stock = newStock;
    return true;
}

// Advance scheduled occurrences to Due or Missed based on RTC time.
void update(uint64_t now) {
    for (size_t i = 0; i < doses_.size(); ++i) {
        DoseOccurrence& dose = doses_[i];
        if (dose.status != DoseStatus::Scheduled && dose.status != DoseStatus::Due) {
            continue;
        }
        if (now < dose.scheduledAt) {
            continue;
        }

        if (now - dose.scheduledAt >= missedGraceSeconds_) {
            dose.status = DoseStatus::Missed;
            dose.missedAlertPending = true;
            missedAlertPending_ = true;
        } else {
            dose.status = DoseStatus::Due;
        }
    }
}

// A cell button dismisses currently pending missed-dose notifications, but
// does not change missed dose history or consume its stock. It then confirms
// at most one active Due occurrence belonging to that same cell.
DoseResult confirmCell(uint8_t cellId, uint64_t now) {
    update(now);
    acknowledgeMissedAlerts();

    if (findCell(cellId) == NULL) {
        return DoseResult::UnknownCell;
    }

    DoseOccurrence* nextDose = NULL;
    for (size_t i = 0; i < doses_.size(); ++i) {
        DoseOccurrence& dose = doses_[i];
        if (dose.cellId != cellId || dose.status != DoseStatus::Due) {
            continue;
        }
        if (nextDose == NULL || dose.scheduledAt < nextDose->scheduledAt) {
            nextDose = &dose;
        }
    }

    if (nextDose == NULL) {
        for (size_t i = 0; i < doses_.size(); ++i) {
            if (doses_[i].cellId == cellId && doses_[i].status == DoseStatus::Missed) {
                return DoseResult::MissedDoseNotCounted;
            }
        }
        return DoseResult::NoDueDose;
    }

    CellInventory* cell = findCell(cellId);
    nextDose->status = DoseStatus::Confirmed;
    if (cell->stock < nextDose->units) {
        cell->stock = 0;
        return DoseResult::ConfirmedStockEmpty;
```

```
        }

        cell->stock = static_cast<uint16_t>(cell->stock - nextDose->units);
        return cell->isLow() ? DoseResult::ConfirmedLowStock : DoseResult::Confirmed;
    }

    // Clears the OLED/notification latch only. Missed statuses remain unchanged.
    void acknowledgeMissedAlerts() {
        missedAlertPending_ = false;
        for (size_t i = 0; i < doses_.size(); ++i) {
            doses_[i].missedAlertPending = false;
        }
    }

    bool hasMissedAlert() const {
        return missedAlertPending_;
    }

    size_t missedDoseCount() const {
        size_t count = 0;
        for (size_t i = 0; i < doses_.size(); ++i) {
            if (doses_[i].status == DoseStatus::Missed) {
                ++count;
            }
        }
        return count;
    }

    const CellInventory* getCell(uint8_t cellId) const {
        return findCell(cellId);
    }

    const DoseOccurrence* getDose(uint16_t doseId) const {
        return findDose(doseId);
    }

    size_t doseCount() const {
        return doses_.size();
    }

    size_t cellCount() const {
        return cells_.size();
    }

private:
    uint32_t missedGraceSeconds_;
    bool missedAlertPending_;
    std::vector<CellInventory> cells_;
    std::vector<DoseOccurrence> doses_;

    CellInventory* findCell(uint8_t cellId) {
        for (size_t i = 0; i < cells_.size(); ++i) {
            if (cells_[i].cellId == cellId) {
                return &cells_[i];
            }
        }
        return NULL;
    }

    const CellInventory* findCell(uint8_t cellId) const {
        for (size_t i = 0; i < cells_.size(); ++i) {
            if (cells_[i].cellId == cellId) {
                return &cells_[i];
            }
        }
        return NULL;
    }

    DoseOccurrence* findDose(uint16_t doseId) {
        for (size_t i = 0; i < doses_.size(); ++i) {
            if (doses_[i].doseId == doseId) {
                return &doses_[i];
            }
```

```
        }
        return NULL;
    }

    const DoseOccurrence* findDose(uint16_t doseId) const {
        for (size_t i = 0; i < doses_.size(); ++i) {
            if (doses_[i].doseId == doseId) {
                return &doses_[i];
            }
        }
        return NULL;
    }
};

#endif
```

A.8 scheduling-event-engine/src/schedule_event_engine.cpp

```
#include "../include/schedule_event_engine.hpp"

ScheduleEventEngine::ScheduleEventEngine()
    : timeProvider(nullptr), gracePeriodMinutes(10), lowStockWarningEnabled(true),
      cellCount(0), dueEventCount(0), missedEventCount(0), callbackCount(0),
      nextEventId(0), firstRun(true)
{
    for (uint8_t i = 0; i < MAX_ENGINE_CELLS; i++) { lastWarnedStock[i] = 0xFFFFU; }
}

ScheduleEventEngine::ScheduleEventEngine(ITimeProvider* provider)
    : timeProvider(provider), gracePeriodMinutes(10), lowStockWarningEnabled(true),
      cellCount(0), dueEventCount(0), missedEventCount(0), callbackCount(0),
      nextEventId(0), firstRun(true)
{
    for (uint8_t i = 0; i < MAX_ENGINE_CELLS; i++) { lastWarnedStock[i] = 0xFFFFU; }
}

void ScheduleEventEngine::setTimeProvider(ITimeProvider* provider)
{
    timeProvider = provider;
}

ITimeProvider* ScheduleEventEngine::getTimeProvider() const
{
    return timeProvider;
}

void ScheduleEventEngine::setGracePeriodMinutes(uint16_t minutes)
{
    gracePeriodMinutes = (minutes > 0) ? minutes : 1;
}

uint16_t ScheduleEventEngine::getGracePeriodMinutes() const
{
    return gracePeriodMinutes;
}

void ScheduleEventEngine::setLowStockWarningEnabled(bool enabled)
{
    lowStockWarningEnabled = enabled;
}

bool ScheduleEventEngine::isLowStockWarningEnabled() const
{
    return lowStockWarningEnabled;
}

bool ScheduleEventEngine::addCell(const EngineCell& cell)
{
    if (cellCount >= MAX_ENGINE_CELLS)
    {
        return false;
    }
    cells[cellCount] = cell;
```

```
    cells[cellCount].cellIndex = cellCount;
    cellCount++;
    return true;
}

bool ScheduleEventEngine::updateCell(uint8_t cellIndex, const EngineCell& cell)
{
    if (cellIndex >= cellCount)
    {
        return false;
    }
    cells[cellIndex] = cell;
    cells[cellIndex].cellIndex = cellIndex;
    return true;
}

void ScheduleEventEngine::clearCells()
{
    cellCount = 0;
    dueEventCount = 0;
    missedEventCount = 0;
}

uint8_t ScheduleEventEngine::getCellCount() const
{
    return cellCount;
}

const EngineCell* ScheduleEventEngine::getCell(uint8_t cellIndex) const
{
    if (cellIndex >= cellCount)
    {
        return nullptr;
    }
    return &cells[cellIndex];
}

EngineCell* ScheduleEventEngine::getCellMutable(uint8_t cellIndex)
{
    if (cellIndex >= cellCount)
    {
        return nullptr;
    }
    return &cells[cellIndex];
}

int16_t ScheduleEventEngine::findCellIndexById(const char* cellId) const
{
    if (!cellId)
    {
        return -1;
    }
    for (uint8_t i = 0; i < cellCount; i++)
    {
        if (strncmp(cells[i].cellId, cellId, MAX_ENGINE_ID_LEN) == 0)
        {
            return static_cast<int16_t>(i);
        }
    }
    return -1;
}

bool ScheduleEventEngine::updateCellStock(uint8_t cellIndex, uint16_t newStock)
{
    if (cellIndex >= cellCount)
    {
        return false;
    }
    cells[cellIndex].currentStock = newStock;
    if (newStock > cells[cellIndex].lowStockThreshold)
    {
        lastWarnedStock[cellIndex] = 0xFFFFU;
    }
```

```
    return true;
}

bool ScheduleEventEngine::triggerDueEvent(uint8_t cellIndex)
{
    if (cellIndex >= cellCount)
    {
        return false;
    }

    if (cells[cellIndex].state != CellState::OCCUPIED || !cells[cellIndex].hasMedicine)
    {
        return false;
    }

    if (cells[cellIndex].currentStock == 0)
    {
        // Out of stock: cannot trigger due dose
        return false;
    }

    if (isCellDue(cellIndex))
    {
        return true; // Already due
    }

    if (dueEventCount >= MAX_ACTIVE_EVENTS)
    {
        return false;
    }

    TimeOfDay currentTime = (timeProvider) ? timeProvider->getCurrentTime() : TimeOfDay();

    DoseEvent event;
    event.eventId = ++nextEventId;
    event.type = EventType::DOSE_DUE;
    event.state = EventState::DUE;

    strncpy(event.cellId, cells[cellIndex].cellId, MAX_ENGINE_ID_LEN - 1);
    event.cellId[MAX_ENGINE_ID_LEN - 1] = '\0';
    event.cellIndex = cellIndex;

    strncpy(event.medicineId, cells[cellIndex].medicineId, MAX_ENGINE_ID_LEN - 1);
    event.medicineId[MAX_ENGINE_ID_LEN - 1] = '\0';

    strncpy(event.medicineName, cells[cellIndex].medicineName, MAX_ENGINE_STRING_LEN - 1);
    event.medicineName[MAX_ENGINE_STRING_LEN - 1] = '\0';

    uint16_t qty = 1;
    if (cells[cellIndex].scheduleCount > 0)
    {
        qty = cells[cellIndex].schedules[0].doseQuantity;
        for (uint8_t s = 0; s < cells[cellIndex].scheduleCount; s++)
        {
            if (cells[cellIndex].schedules[s].hour == currentTime.hour &&
                cells[cellIndex].schedules[s].minute == currentTime.minute)
            {
                qty = cells[cellIndex].schedules[s].doseQuantity;
                break;
            }
        }
    }
    event.doseQuantity = (qty > 0) ? qty : 1;
    event.scheduledTime = currentTime;
    event.triggeredTime = currentTime;
    event.triggeredEpoch = (timeProvider) ? timeProvider->getEpochSeconds() : 0;

    activeDueEvents[dueEventCount++] = event;
    notifyListeners(event);
    return true;
}

bool ScheduleEventEngine::registerEventCallback(EventCallbackFn callback, void* userData)
```

```
{
    if (!callback || callbackCount >= MAX_EVENT_LISTENERS)
    {
        return false;
    }
    for (uint8_t i = 0; i < callbackCount; i++)
    {
        if (callbacks[i].fn == callback && callbacks[i].userData == userData)
        {
            return true; // Already registered
        }
    }
    callbacks[callbackCount].fn = callback;
    callbacks[callbackCount].userData = userData;
    callbackCount++;
    return true;
}

bool ScheduleEventEngine::unregisterEventCallback(EventCallbackFn callback)
{
    for (uint8_t i = 0; i < callbackCount; i++)
    {
        if (callbacks[i].fn == callback)
        {
            for (uint8_t j = i; j < callbackCount - 1; j++)
            {
                callbacks[j] = callbacks[j + 1];
            }
            callbackCount--;
            return true;
        }
    }
    return false;
}

void ScheduleEventEngine::notifyListeners(const DoseEvent& event)
{
    for (uint8_t i = 0; i < callbackCount; i++)
    {
        if (callbacks[i].fn)
        {
            callbacks[i].fn(event, callbacks[i].userData);
        }
    }
}

uint32_t ScheduleEventEngine::calculateElapsedMinutes(const TimeOfDay& start, const TimeOfDay& end)
const
{
    uint32_t startMin = start.toMinutesOfDay();
    uint32_t endMin = end.toMinutesOfDay();

    if (endMin >= startMin)
    {
        return endMin - startMin;
    }
    // Midnight rollover case
    return (1440UL - startMin) + endMin;
}

void ScheduleEventEngine::checkMidnightRollover(const TimeOfDay& currentTime)
{
    if (firstRun)
    {
        return;
    }

    // Detected transition across midnight (e.g., from 23:59 to 00:00)
    if (currentTime.hour < lastEvaluatedTime.hour)
    {
        resetDailyTriggerFlags();

        DoseEvent resetEvent;
```

```
        resetEvent.eventId = ++nextEventId;
        resetEvent.type = EventType::DAILY_RESET;
        resetEvent.state = EventState::IDLE;
        resetEvent.triggeredTime = currentTime;
        notifyListeners(resetEvent);
    }
}

void ScheduleEventEngine::resetDailyTriggerFlags()
{
    for (uint8_t i = 0; i < cellCount; i++)
    {
        for (uint8_t j = 0; j < cells[i].scheduleCount; j++)
        {
            cells[i].schedules[j].triggeredToday = false;
        }
    }
}

void ScheduleEventEngine::checkSchedules(const TimeOfDay& currentTime)
{
    // Avoid evaluating trigger within the exact same minute unless first run
    bool isSameMinute = (!firstRun &&
                         currentTime.hour == lastEvaluatedTime.hour &&
                         currentTime.minute == lastEvaluatedTime.minute);

    if (isSameMinute)
    {
        return;
    }

    for (uint8_t i = 0; i < cellCount; i++)
    {
        if (cells[i].state != CellState::OCCUPIED || !cells[i].hasMedicine)
        {
            continue;
        }

        for (uint8_t j = 0; j < cells[i].scheduleCount; j++)
        {
            DoseSchedule& sched = cells[i].schedules[j];

            if (!sched.enabled || sched.triggeredToday)
            {
                continue;
            }

            if (currentTime.hour == sched.hour && currentTime.minute == sched.minute)
            {
                sched.triggeredToday = true;

                if (cells[i].currentStock == 0)
                {
                    // OUT OF STOCK: cannot dispense dose from empty cell!
                    DoseEvent emptyEvent;
                    emptyEvent.eventId = ++nextEventId;
                    emptyEvent.type = EventType::LOW_STOCK_WARNING;
                    emptyEvent.state = EventState::IDLE;
                    strncpy(emptyEvent.cellId, cells[i].cellId, MAX_ENGINE_ID_LEN - 1);
                    emptyEvent.cellId[MAX_ENGINE_ID_LEN - 1] = '\0';
                    emptyEvent.cellIndex = i;
                    strncpy(emptyEvent.medicineId, cells[i].medicineId, MAX_ENGINE_ID_LEN - 1);
                    emptyEvent.medicineId[MAX_ENGINE_ID_LEN - 1] = '\0';
                    strncpy(emptyEvent.medicineName, cells[i].medicineName, MAX_ENGINE_STRING_LEN -
1);
                    emptyEvent.medicineName[MAX_ENGINE_STRING_LEN - 1] = '\0';
                    emptyEvent.doseQuantity = 0; // Indicates 0 stock
                    emptyEvent.triggeredTime = currentTime;
                    emptyEvent.scheduledTime = TimeOfDay(sched.hour, sched.minute);
                    notifyListeners(emptyEvent);
                    continue;
                }
```

```
                if (dueEventCount < MAX_ACTIVE_EVENTS)
                {
                    DoseEvent event;
                    event.eventId = ++nextEventId;
                    event.type = EventType::DOSE_DUE;
                    event.state = EventState::DUE;

                    strncpy(event.cellId, cells[i].cellId, MAX_ENGINE_ID_LEN - 1);
                    event.cellId[MAX_ENGINE_ID_LEN - 1] = '\0';
                    event.cellIndex = i;

                    strncpy(event.medicineId, cells[i].medicineId, MAX_ENGINE_ID_LEN - 1);
                    event.medicineId[MAX_ENGINE_ID_LEN - 1] = '\0';

                    strncpy(event.medicineName, cells[i].medicineName, MAX_ENGINE_STRING_LEN - 1);
                    event.medicineName[MAX_ENGINE_STRING_LEN - 1] = '\0';

                    event.doseQuantity = sched.doseQuantity;
                    event.scheduledTime = TimeOfDay(sched.hour, sched.minute);
                    event.triggeredTime = currentTime;
                    event.triggeredEpoch = (timeProvider) ? timeProvider->getEpochSeconds() : 0;

                    activeDueEvents[dueEventCount++] = event;
                    notifyListeners(event);
                }
            }
        }
    }
}

void ScheduleEventEngine::checkGracePeriods(const TimeOfDay& currentTime)
{
    for (int16_t i = static_cast<int16_t>(dueEventCount) - 1; i >= 0; i--)
    {
        uint32_t elapsed = calculateElapsedMinutes(activeDueEvents[i].scheduledTime, currentTime);

        if (elapsed >= gracePeriodMinutes)
        {
            // Transition from DUE to MISSED
            DoseEvent missedEvent = activeDueEvents[i];
            missedEvent.type = EventType::DOSE_MISSED;
            missedEvent.state = EventState::MISSED;
            missedEvent.triggeredTime = currentTime;

            // Remove from due list
            removeDueEventByIndex(static_cast<uint8_t>(i));

            // Add to missed list
            if (missedEventCount < MAX_MISSED_EVENTS)
            {
                activeMissedEvents[missedEventCount++] = missedEvent;
            }

            notifyListeners(missedEvent);
        }
    }
}

void ScheduleEventEngine::checkLowStock()
{
    if (!lowStockWarningEnabled)
    {
        return;
    }

    for (uint8_t i = 0; i < cellCount; i++)
    {
        if (!cells[i].hasMedicine)
        {
            continue;
        }

        const uint16_t stock = cells[i].currentStock;
```

```
        if (stock <= cells[i].lowStockThreshold)
        {
            // Only fire once per stock level; avoids spamming every update() tick.
            if (stock == lastWarnedStock[i])
            {
                continue;
            }
            lastWarnedStock[i] = stock;

            DoseEvent warnEvent;
            warnEvent.eventId = ++nextEventId;
            warnEvent.type = EventType::LOW_STOCK_WARNING;
            warnEvent.state = EventState::IDLE;

            strncpy(warnEvent.cellId, cells[i].cellId, MAX_ENGINE_ID_LEN - 1);
            warnEvent.cellId[MAX_ENGINE_ID_LEN - 1] = '\0';
            warnEvent.cellIndex = i;

            strncpy(warnEvent.medicineId, cells[i].medicineId, MAX_ENGINE_ID_LEN - 1);
            warnEvent.medicineId[MAX_ENGINE_ID_LEN - 1] = '\0';

            strncpy(warnEvent.medicineName, cells[i].medicineName, MAX_ENGINE_STRING_LEN - 1);
            warnEvent.medicineName[MAX_ENGINE_STRING_LEN - 1] = '\0';

            warnEvent.doseQuantity = stock;
            warnEvent.triggeredTime = (timeProvider) ? timeProvider->getCurrentTime() : TimeOfDay();

            notifyListeners(warnEvent);
        }
        else
        {
            // Stock is above threshold; reset guard so a future drop triggers again.
            lastWarnedStock[i] = 0xFFFFU;
        }
    }
}

void ScheduleEventEngine::update()
{
    if (!timeProvider || !timeProvider->isAvailable())
    {
        return;
    }

    TimeOfDay currentTime = timeProvider->getCurrentTime();

    checkMidnightRollover(currentTime);
    checkSchedules(currentTime);
    checkGracePeriods(currentTime);
    checkLowStock();   // Fires LOW_STOCK_WARNING when stock <= threshold (once per stock level)

    lastEvaluatedTime = currentTime;
    firstRun = false;
}

bool ScheduleEventEngine::confirmDose(uint8_t cellIndex)
{
    for (uint8_t i = 0; i < dueEventCount; i++)
    {
        if (activeDueEvents[i].cellIndex == cellIndex)
        {
            DoseEvent confirmedEvent = activeDueEvents[i];
            confirmedEvent.type = EventType::DOSE_CONFIRMED;
            confirmedEvent.state = EventState::CONFIRMED;
            confirmedEvent.resolvedTime = (timeProvider) ? timeProvider->getCurrentTime() :
TimeOfDay();

            // Decrement stock in cell (clamped at 0)
            if (cells[cellIndex].currentStock > 0)
            {
                if (cells[cellIndex].currentStock >= confirmedEvent.doseQuantity)
                {
```

```
                    cells[cellIndex].currentStock -= confirmedEvent.doseQuantity;
                }
                else
                {
                    cells[cellIndex].currentStock = 0;
                }
            }

            removeDueEventByIndex(i);
            notifyListeners(confirmedEvent);
            return true;
        }
    }
    return false;
}

bool ScheduleEventEngine::confirmDose(const char* cellId)
{
    int16_t index = findCellIndexById(cellId);
    if (index >= 0)
    {
        return confirmDose(static_cast<uint8_t>(index));
    }
    return false;
}

bool ScheduleEventEngine::dismissMissedDose(uint8_t cellIndex)
{
    for (uint8_t i = 0; i < missedEventCount; i++)
    {
        if (activeMissedEvents[i].cellIndex == cellIndex)
        {
            DoseEvent dismissedEvent = activeMissedEvents[i];
            dismissedEvent.type = EventType::DOSE_DISMISSED;
            dismissedEvent.state = EventState::DISMISSED;
            dismissedEvent.resolvedTime = (timeProvider) ? timeProvider->getCurrentTime() :
TimeOfDay();

            removeMissedEventByIndex(i);
            notifyListeners(dismissedEvent);
            return true;
        }
    }
    return false;
}

bool ScheduleEventEngine::dismissMissedDose(const char* cellId)
{
    int16_t index = findCellIndexById(cellId);
    if (index >= 0)
    {
        return dismissMissedDose(static_cast<uint8_t>(index));
    }
    return false;
}

uint8_t ScheduleEventEngine::dismissAllMissedDoses()
{
    uint8_t cleared = 0;
    while (missedEventCount > 0)
    {
        DoseEvent dismissedEvent = activeMissedEvents[0];
        dismissedEvent.type = EventType::DOSE_DISMISSED;
        dismissedEvent.state = EventState::DISMISSED;
        dismissedEvent.resolvedTime = (timeProvider) ? timeProvider->getCurrentTime() : TimeOfDay();

        removeMissedEventByIndex(0);
        notifyListeners(dismissedEvent);
        cleared++;
    }
    return cleared;
}
```

```
bool ScheduleEventEngine::handleCellButtonPress(uint8_t cellIndex)
{
    // Case 1: Active DUE dose for this cell -> Confirm it
    if (confirmDose(cellIndex))
    {
        return true;
    }

    // Case 2: Active MISSED dose for this cell -> Dismiss reminder
    if (dismissMissedDose(cellIndex))
    {
        return true;
    }

    // Case 3: Any button press clears all missed reminders (Mobile Clock reminder style)
    if (missedEventCount > 0)
    {
        dismissAllMissedDoses();
        return true;
    }

    return false;
}

bool ScheduleEventEngine::removeDueEventByIndex(uint8_t index)
{
    if (index >= dueEventCount)
    {
        return false;
    }
    for (uint8_t i = index; i < dueEventCount - 1; i++)
    {
        activeDueEvents[i] = activeDueEvents[i + 1];
    }
    dueEventCount--;
    return true;
}

bool ScheduleEventEngine::removeMissedEventByIndex(uint8_t index)
{
    if (index >= missedEventCount)
    {
        return false;
    }
    for (uint8_t i = index; i < missedEventCount - 1; i++)
    {
        activeMissedEvents[i] = activeMissedEvents[i + 1];
    }
    missedEventCount--;
    return true;
}

uint8_t ScheduleEventEngine::getDueEventCount() const
{
    return dueEventCount;
}

const DoseEvent* ScheduleEventEngine::getDueEvent(uint8_t index) const
{
    if (index >= dueEventCount)
    {
        return nullptr;
    }
    return &activeDueEvents[index];
}

const DoseEvent* ScheduleEventEngine::getDueEventForCell(uint8_t cellIndex) const
{
    for (uint8_t i = 0; i < dueEventCount; i++)
    {
        if (activeDueEvents[i].cellIndex == cellIndex)
        {
            return &activeDueEvents[i];
```

```
        }
    }
    return nullptr;
}

bool ScheduleEventEngine::hasActiveDueEvents() const
{
    return dueEventCount > 0;
}

bool ScheduleEventEngine::isCellDue(uint8_t cellIndex) const
{
    return getDueEventForCell(cellIndex) != nullptr;
}

uint8_t ScheduleEventEngine::getMissedEventCount() const
{
    return missedEventCount;
}

const DoseEvent* ScheduleEventEngine::getMissedEvent(uint8_t index) const
{
    if (index >= missedEventCount)
    {
        return nullptr;
    }
    return &activeMissedEvents[index];
}

const DoseEvent* ScheduleEventEngine::getMissedEventForCell(uint8_t cellIndex) const
{
    for (uint8_t i = 0; i < missedEventCount; i++)
    {
        if (activeMissedEvents[i].cellIndex == cellIndex)
        {
            return &activeMissedEvents[i];
        }
    }
    return nullptr;
}

bool ScheduleEventEngine::hasActiveMissedEvents() const
{
    return missedEventCount > 0;
}

bool ScheduleEventEngine::isCellMissed(uint8_t cellIndex) const
{
    return getMissedEventForCell(cellIndex) != nullptr;
}

bool ScheduleEventEngine::getNextScheduledDose(TimeOfDay& outTime, uint8_t& outCellIndex) const
{
    if (!timeProvider || !timeProvider->isAvailable())
    {
        return false;
    }

    TimeOfDay now = timeProvider->getCurrentTime();
    uint32_t currentMin = now.toMinutesOfDay();
    uint32_t minDiff = 1441; // Greater than 24 hours in minutes
    bool found = false;

    for (uint8_t i = 0; i < cellCount; i++)
    {
        if (cells[i].state != CellState::OCCUPIED || !cells[i].hasMedicine)
        {
            continue;
        }

        for (uint8_t j = 0; j < cells[i].scheduleCount; j++)
        {
            const DoseSchedule& sched = cells[i].schedules[j];
```

```
            if (!sched.enabled)
            {
                continue;
            }

            uint32_t schedMin = static_cast<uint32_t>(sched.hour) * 60UL + sched.minute;
            uint32_t diff = 0;

            if (schedMin > currentMin)
            {
                diff = schedMin - currentMin;
            }
            else
            {
                // Next day schedule
                diff = (1440UL - currentMin) + schedMin;
            }

            if (diff < minDiff)
            {
                minDiff = diff;
                outTime = TimeOfDay(sched.hour, sched.minute);
                outCellIndex = i;
                found = true;
            }
        }
    }
    return found;
}
```

Appendix B: Result Screenshots

Add here any result screenshots that are not already shown in Sections 5 and 6, for example extra test cases, terminal logs or debugger views.

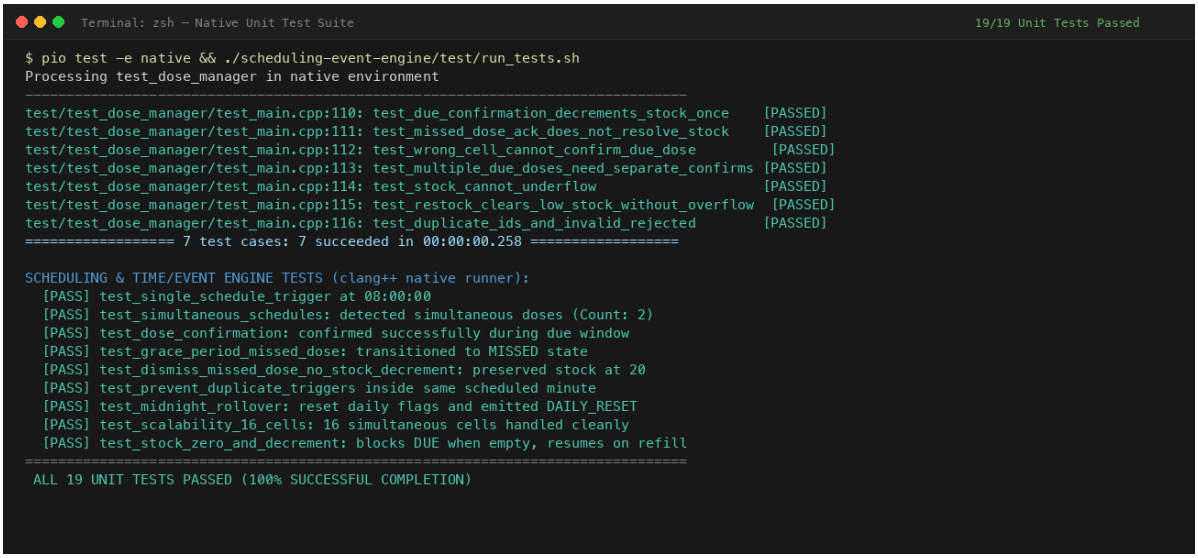


Figure B1. Automated unit test execution log (Dose Manager & Scheduling Engine).

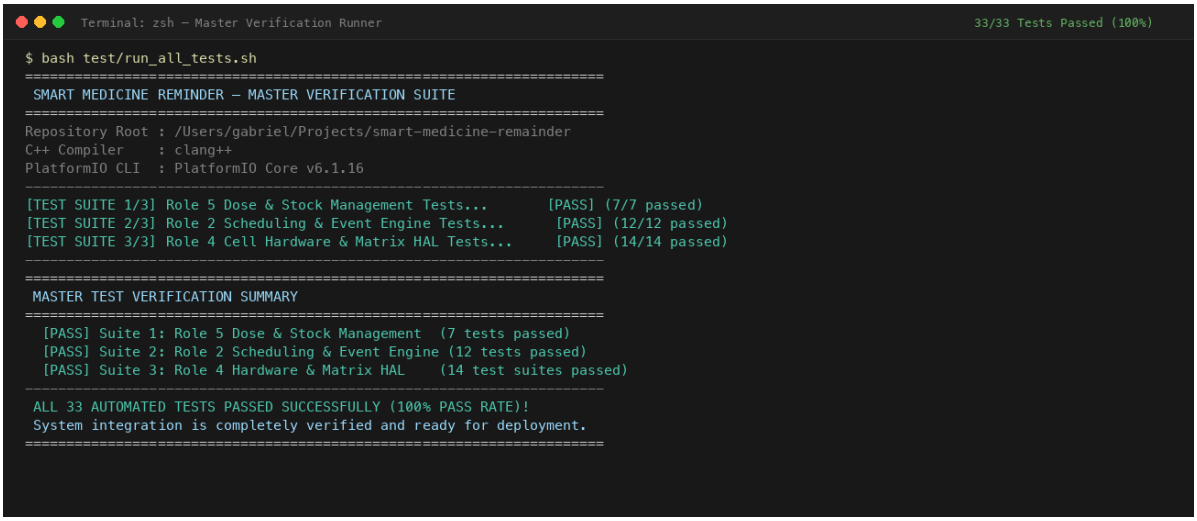


Figure B2. Master test suite execution log and matrix verification summary.